

绪论

从计算，到算法

01-A

工欲善其事，必先利其器

君子生非異也，善假於物也

Computer science should be called computing science, for the same reason why surgery is not called knife science.

- E. Dijkstra

邓俊辉

deng@tsinghua.edu.cn

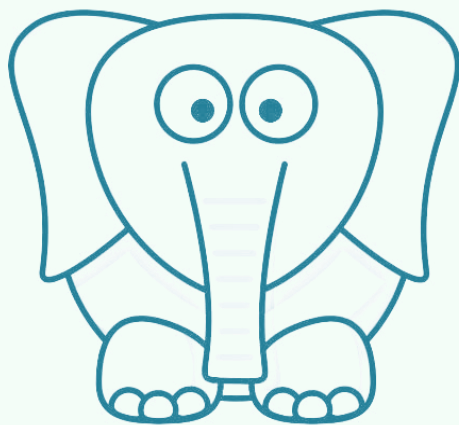
曹冲称象：船、水、石

置象大船之上，而

刻其水痕所至

称物以载之，则

校可知矣



鄧哀王冲字倉舒。少聰察岐嶷，生五六歲，智意所及，有若成人之智。時孫權曾致巨象，太祖欲知其斤重，訪之羣下，咸莫能出其理。冲曰：「置象大船之上，而刻其水痕所至，稱物以載之，則校可知矣。」太祖大悅，即施行焉。時軍國多事，用刑嚴重。太祖馬鞍在

欧几里得：尺规计算机

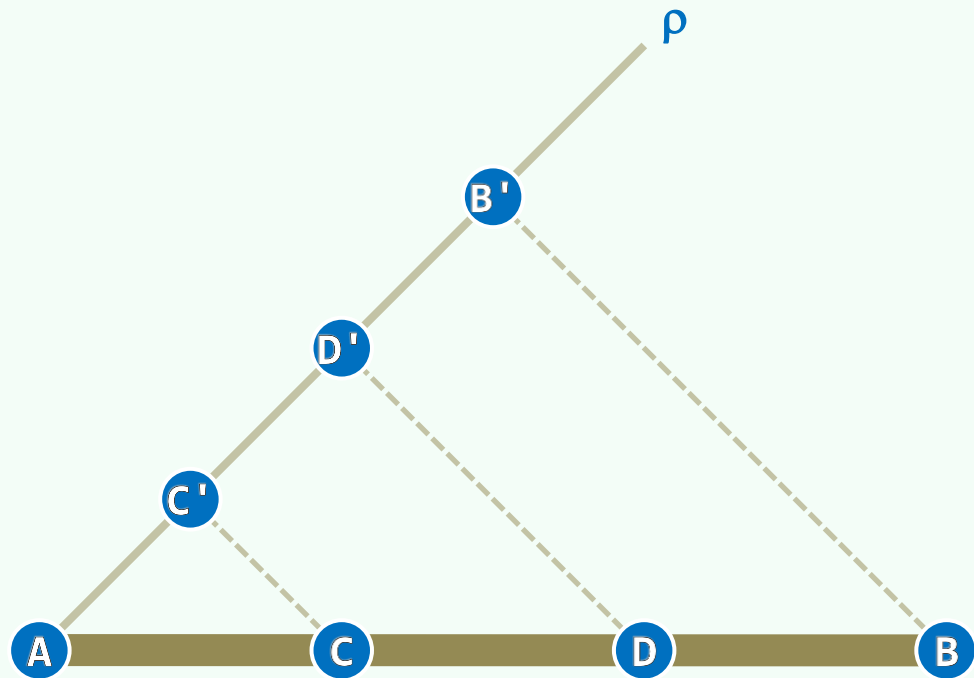
❖ 任给平面上线段 AB （输入），将其三等分（输出）

- 从 A 发出一条与 AB 不重合的射线 ρ
- 在 ρ 上取 $|AC'| = |C'D'| = |D'B'|$
- 联接 $B'B$
- 经 D' 做 $B'B$ 的平行线，交 AB 于 D
- 经 C' 做 $B'B$ 的平行线，交 AB 于 C

❖ 这里的计算机是什么？

❖ 它能够解决什么问题？不能解决什么问题？

❖ 子程序：过直线外一点，做平行线



算法的要素

❖ 计算 = 信息处理 = 借助某种工具，遵照一定规则，以明确而机械的形式进行

❖ 计算模型 = 计算机 = 信息处理工具

❖ 所谓算法，即特定计算模型下，旨在解决特定问题的指令序列

input 输入	待处理的信息（问题）
output 输出	经处理的信息（答案）
correctness 正确	的确可以解决指定的问题
definiteness 明确	可描述为一个由基本操作组成的序列 //加盐少许？ 加糖适量？ 煮至半熟？
effectiveness 可行	每一基本操作都可实现，且在常数时间内完成 //把大象放进冰箱？
finiteness 有穷	对于任何输入，经有穷次基本操作，都可以得到输出
...	...

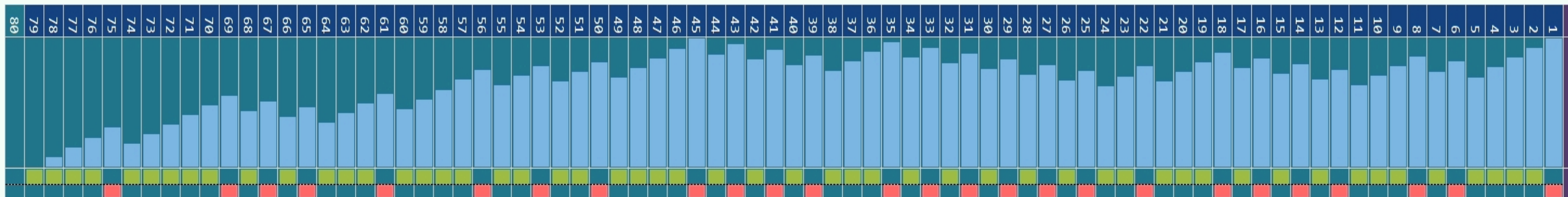
有穷性: Hailstone序列

$$Hailstone(n) = \begin{cases} \{1\} & (n \leq 1) \\ \{n\} \cup Hailstone(n/2) & (n \text{ is even}) \\ \{n\} \cup Hailstone(3n + 1) & (n \text{ is odd}) \end{cases}$$

42, 21, 64, 32, ..., 1

7, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, ..., 1

27, 82, 41, 124, 62, 31, 94, 47, 142, 71, 214, 107, ...



有穷性：程序~算法

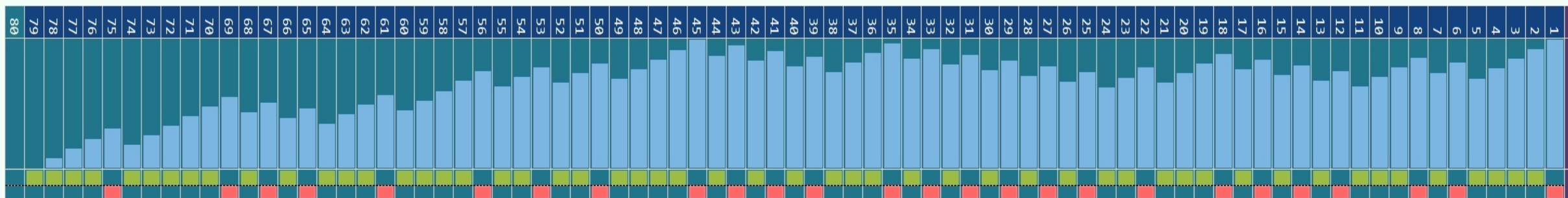
```
❖ int hailstone( int n ) {  
    int length = 1;  
    for ( ; 1 < n; length++ )  
        n = n%2 ? 3*n+1 : n/2;  
    return length;  
}
```

❖ 对于任意的 $n > 0$

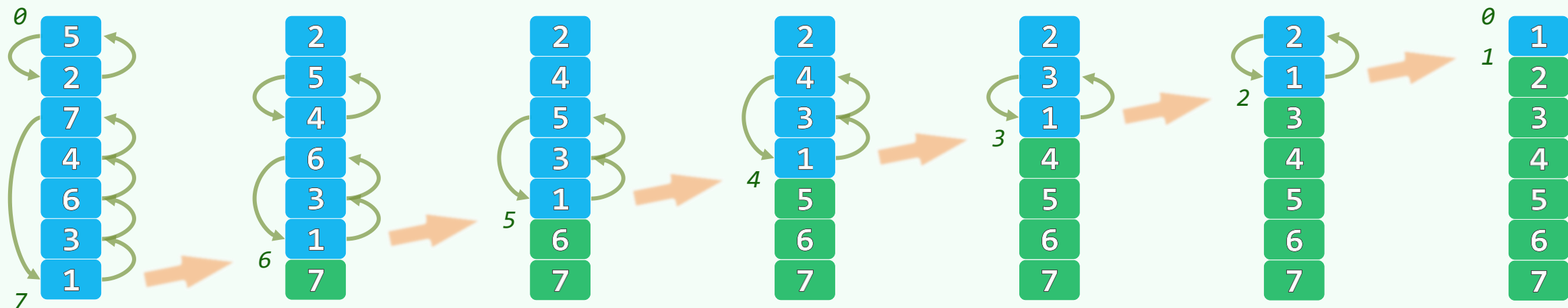
总有 $|\text{Hailstone}(n)| < \infty$?

❖ Erdos:

Mathematics is **NOT** yet ready
for such problems



起泡排序：反复做起泡扫描



```
❖ void bubbleSort( int A[], int n ) {
```

```
    while ( 1 < n-- ) //做n-1趟起泡扫描：有序序列中，每一对相邻元素都有序
```

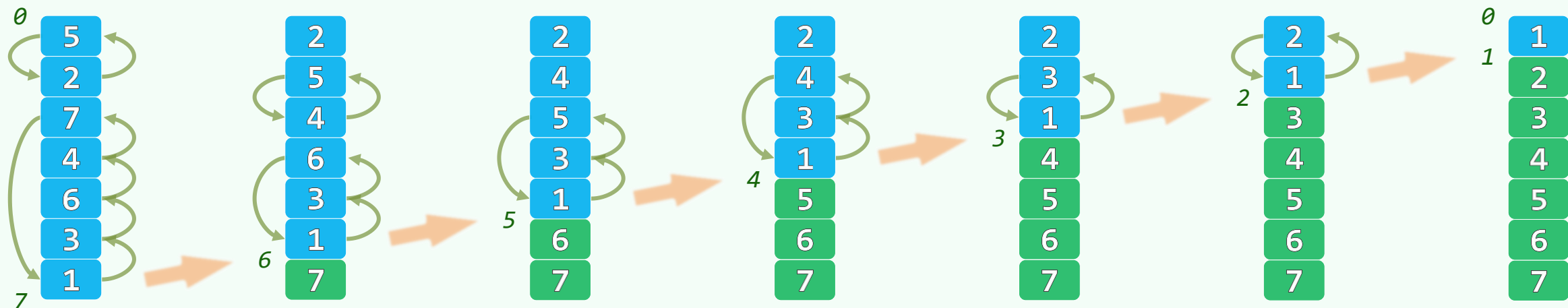
```
        for ( int i = 0; i < n; i++ ) //自前向后地逐对检视每对相邻元素
```

```
            if ( A[i] > A[i + 1] ) //一旦逆序，则
```

```
                swap( A[i], A[i + 1] ); //交换之
```

```
    } //bubblesort
```

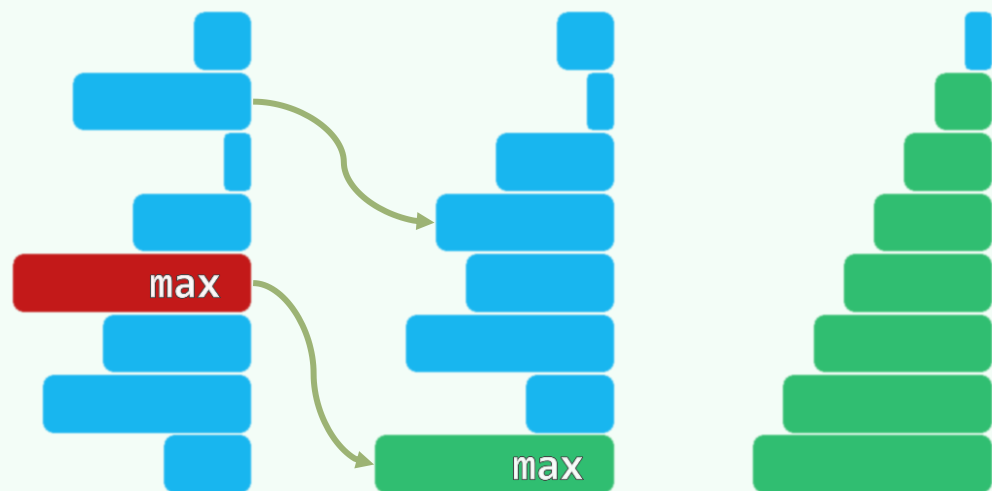
起泡排序：因不变，而正确



❖ 序列模式：未必有序的前缀 $[0, s)$ + 已然有序的后缀 $[s, n)$

❖ 初始 $s = n$ ；随后，每经过一趟起泡扫描

- [实质元] 前缀中的 $\max$ ，必然就位
- [不变性] 序列模式依然保持，且
- [单调性] 有序后缀都会增长



从正确的算法，到“好”的算法

❖ 语法正确，能够编译、链接

❖ 功能正确，能够处理：

简单的输入、大规模的输入

一般性的输入、退化的输入

任意合法的输入

❖ 健壮：能辨别不合法的输入并做适当处理
而不致非正常退出

❖ 可读、易维护：结构化 + 命名 + 注释 + ...

❖ 效率：速度尽可能快
存储空间尽可能少

Algorithms + Data Structures = Programs

(Algorithms + Data Structures) x Efficiency = Computation

绪论

计算模型：图灵机

To measure is to know. If you can not measure it,
you can not improve it. - Lord Kelvin

洞察一个对象的形式性质，把它同它的质料部分完全分开，然后沉思它，
然后判断时间，即这一特殊形式的事物自然要持续的最长时间

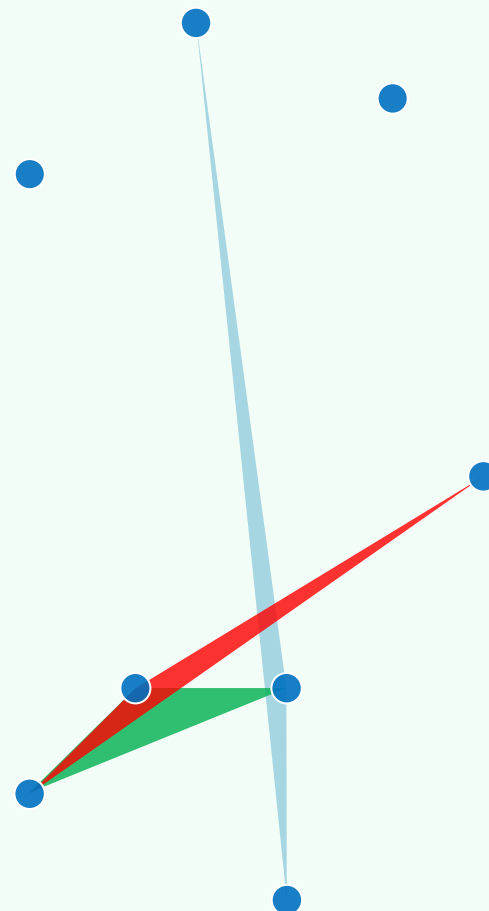
Sometimes it is the people no one can imagine anything of
who do the things no one can imagine. - A. Turing

邓俊辉

deng@tsinghua.edu.cn

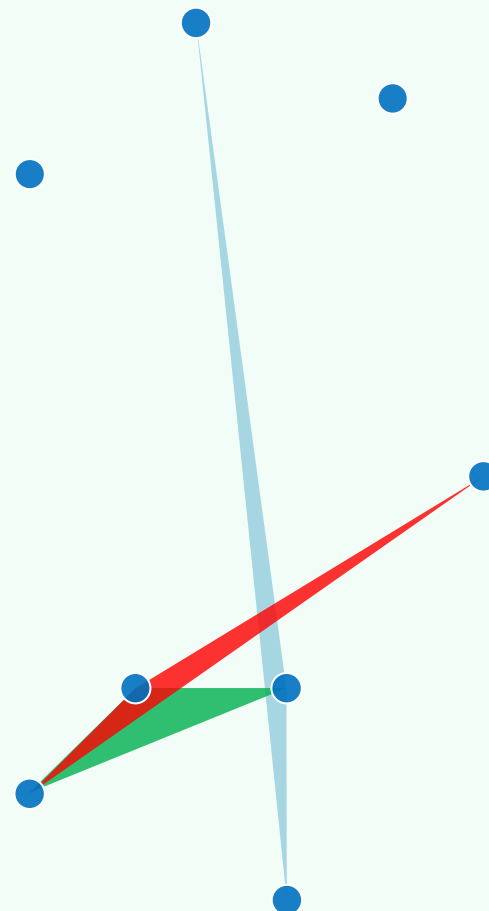
特定算法 + 不同实例

- ❖ 令: $T_A(n)$ = 用算法A求解同一问题规模为n的实例, 所需的计算成本
讨论特定算法A (及其对应的问题) 时, 可简记作 $T(n)$
- ❖ 然而: 同等规模问题实例的计算成本不尽相同, 甚至差异极大
- ❖ 例如: 任给平面上n个点, 在它们定义的 $\binom{n}{3}$ 个三角形中,
是否某一个的面积不超过 5.0 ?
- ❖ 蛮力: 最坏情况下需枚举所有三角形; 但运气好的话...
- ❖ 既然如此, 又该如何定义 $T(n)$ 呢?
- ❖ 稳妥起见, 取 $T(n) = \max\{ T(P) \mid |P| = n \}$
亦即, 在规模同为n的所有实例中, 只关注最坏 (成本最高) 者

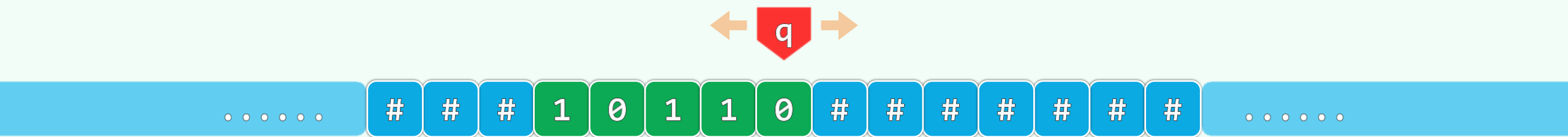


特定问题 + 不同算法

- ❖ 同一问题通常有多种算法，如何**评判**其优劣？
- ❖ 实验统计是最直接的方法，但并不足以准确反映算法的真正效率
 - 不同的算法，可能更适应于不同**规模**的输入
 - 不同的算法，可能更适应于不同**类型**的输入
 - 同一算法，可能由不同**程序员**、用不同**程序语言**、经不同**编译器**生成
 - 同一算法，可能实现并运行于不同的**体系结构**、**操作系统**...
- ❖ 为给出**客观**的评判，需要抽象出一个**理想**的平台或模型
 - 不再依赖于上述种种具体的因素
 - 从而直接而准确地描述、测量并评价算法



TM: 构成部件



❖ Tape ~ 纷繁复杂的大千世界

依次均匀地划分为单元 (Cell)

各存有某一符号, 初始均为 '#'

❖ State ~ 情绪

- 读写头 (TM) 总是处于某一状态
- 每经一拍, 可按**规则**转向另一种状态

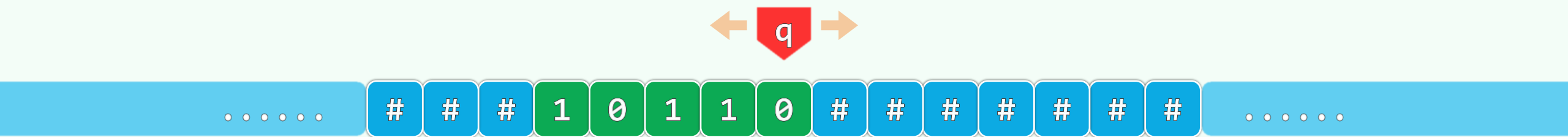
❖ Head ~ 万物之灵

- 总是对准某一单元, 并可读取或改写其中的字符
- 每经一拍, 可左移或右移一个单元

❖ Alphabet ~ 语言

- 使用有限种符号

TM: 转换函数



- ❖ Transition Function:
($q, c; d, L/R, p$)
- ❖ 若当前状态为 q , 且当前字符为 c , 则
 - 将当前字符改写为 d
 - 转向左/右侧邻格
 - 转入 p 状态
- ❖ Sequential Access: 每次只能移动一步
- ❖ 特别地, 一旦转入状态 h , 便随即停机
- ❖ 从启动至停机, 所经历的节拍数目
即可用以度量计算时间 $T(n)$
- ❖ 亦等于Head累计的移动步数 (或距离)

实例：Increase

❖ 功能：将二进制非负整数加一

❖ 原理：全'1'的后缀，翻转为全'0'

原最低位'0'或'#'翻转为'1'

❖ ($\leftarrow$, 1; 0, L, $\leftarrow$) //左行, 1 $\rightarrow$ 0

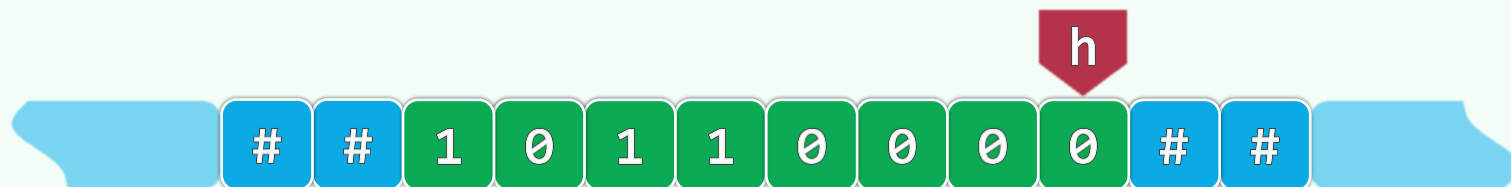
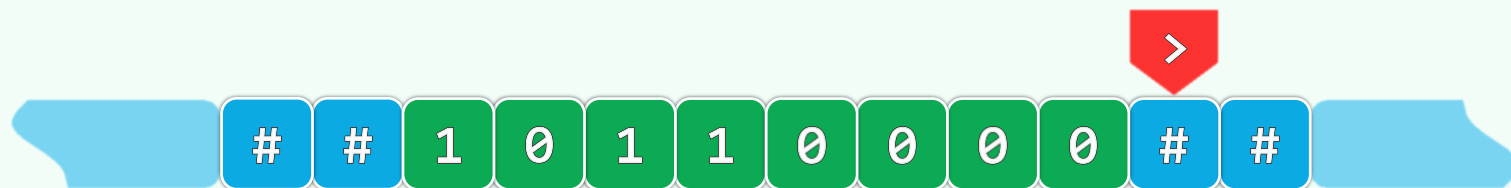
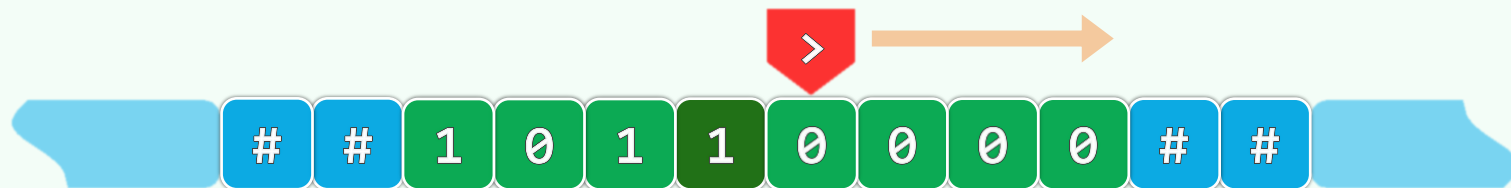
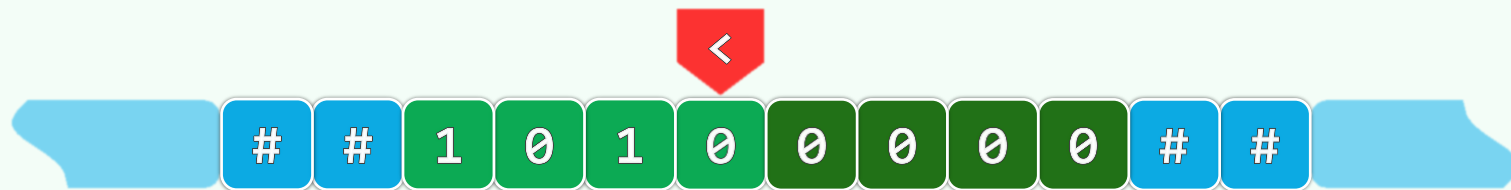
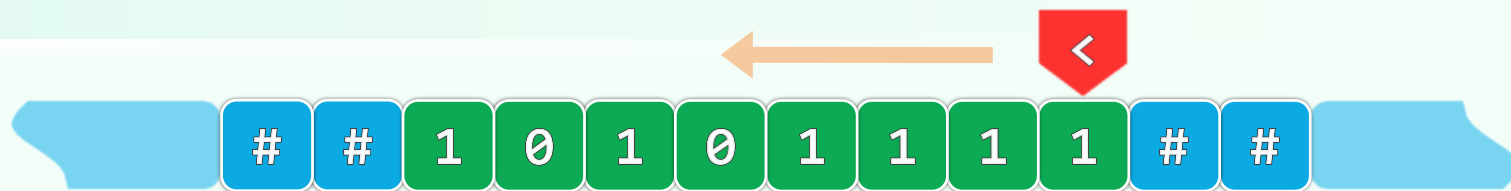
($\leftarrow$, 0; 1, R, $\rightarrow$) //掉头, 0 $\rightarrow$ 1

($\leftarrow$, #; 1, R, $\rightarrow$) //可否省略?

($\rightarrow$, 0; 0, R, $\rightarrow$) //右行

($\rightarrow$, #; #, L, h) //?

❖ $T(n) = T(8) = 4 + 5 + 1 = 10$



绪论

计算模型：RAM

01-B2

三年之喪，二十五月而畢，哀痛未盡，思慕未忘，然而禮以是斷之者，豈不以送死有已，復生有節也哉！

当有加减施加拳脚的地方，理性便有了容身之处；而在加减无所适从的地方，理性也就失去了容身之所

邓俊辉

deng@tsinghua.edu.cn

Random Access Machine: 组成 + 语言



❖ 寄存器顺序编号，总数没有限制

//但愿如此

❖ 可通过编号**直接访问**任意寄存器

//call-by-rank

❖ IR = Instruction Register

//指示当前正在执行哪条指令

❖ 每一**基本操作**仅需常数时间

$R[i] \leftarrow c$ GOTO #

$R[i] \leftarrow R[j]$

$R[i] \leftarrow R[R[j]]$ IF $R[i] = 0$ GOTO #

$R[R[i]] \leftarrow R[j]$ IF $R[i] > 0$ GOTO #

$R[i] \leftarrow R[j] + R[k]$

$R[i] \leftarrow R[j] - R[k]$ STOP

Random Access Machine: 效率



- ❖ 与TM模型一样，RAM模型也是一般计算工具的简化与抽象
使我们可以**独立于**具体的平台，对算法的效率做出**可信**度量与**中肯**评判
- ❖ 在这些模型中
 - 算法的**运行时间** $\propto$ 算法需要执行的基本**操作次数**
 - $T(n)$ = 算法为求解规模为n的问题，所需执行的基本操作次数
- ❖ 思考：在TM、RAM等模型中衡量算法效率，为何通常只需考查运行时间？空间呢？

实例：Ceiling Division：思路

❖ $\forall c \geq 0$ and $d > 0$, define

$$\begin{aligned}\lceil c/d \rceil &= \min\{ x \in \mathbb{Z} \mid c \leq d \cdot x \} \\ &= \min\{ x \mid c - 1 < d \cdot x \}\end{aligned}$$

❖ 例如: $\lceil 2/7 \rceil = 1$ $\lceil 2025/46 \rceil = 45$

$$\lceil 35/5 \rceil = 7 \qquad \lceil 2025/45 \rceil = 45$$

$$\lceil 41/7 \rceil = 6 \qquad \lceil 2025/44 \rceil = 47$$

❖ 思路：反复地从 $R[0] = c$ 中，减去 $R[1] = d$

统计在下溢之前，所做减法的次数x

[illegible]

实例：Ceiling Division: 算法

```
[0] R[3] <- 1 //constant increment
[1] GOTO 4 //in case R[0] = c == 0
[2] R[2] <- R[2] + R[3] //x++
[3] R[0] <- R[0] - R[1] //c -= d
[4] IF R[0] > 0 GOTO 2 //if c > 0 goto 2
[5] R[0] <- R[2] //else copy x to R[0] and
[6] STOP //return R[0] = x = ⌈c/d⌋
```

❖ 时间成本 = 快照表的高度 = 各条指令累计的执行次数

$$T(25,12) = 1 + 1 + 3 + 3 + 4 + 1 + 1 = 14$$

Step	IR	R[0]	R[1]	R[2]	R[3]
0	[0]	25	12	0	0
1	[1]	^	^	^	1
2	[4]	^	^	^	^
3	[2]	^	^	^	^
4	[3]	^	^	1	^
5	[4]	13	^	^	^
6	[2]	^	^	^	^
7	[3]	^	^	2	^
8	[4]	1	^	^	^
9	[2]	^	^	^	^
10	[3]	^	^	3	^
11	[4]	-11	^	^	^
12	[5]	^	^	^	^
13	[6]	3	^	^	^

绪论

渐近复杂度：大 θ 记号

好讀書，不求甚解；每有會意，便欣然忘食

Any time you are stuck on a problem, introduce more notation. - Chris Skinner

Mathematics is more in need of good notations than of new theorems. - A. Turing

邓俊辉

deng@tsinghua.edu.cn

渐近分析

❖ 回到原先的问题:

随着问题规模的增长, 计算成本如何增长?

❖ 这里更关心:

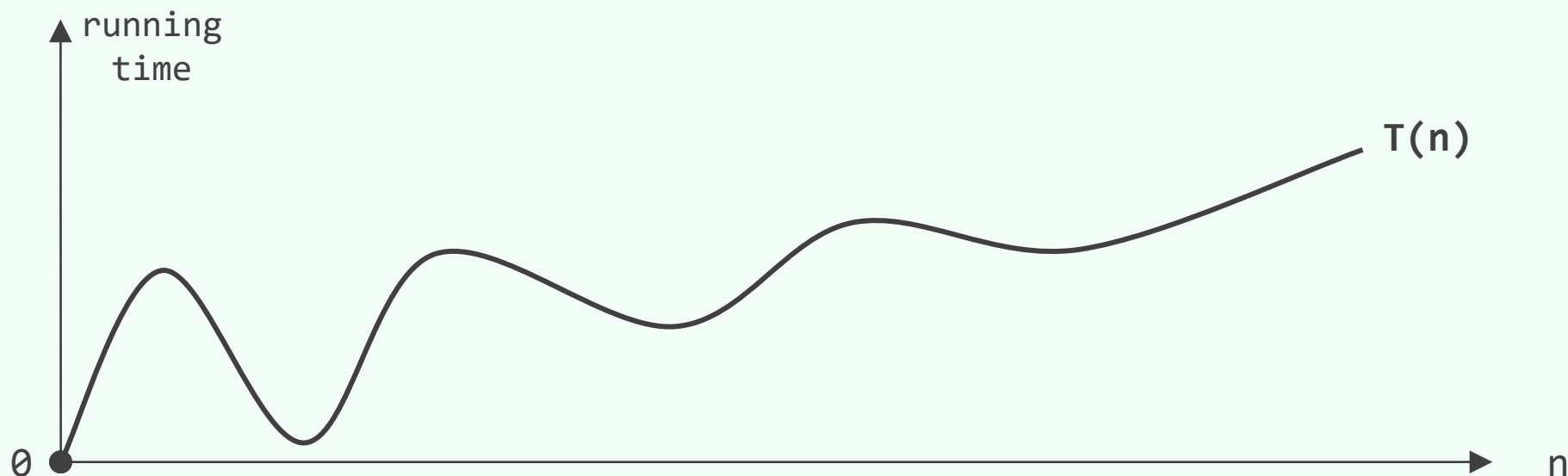
问题规模**足够大**之后, 计算成本的**增长趋势**

❖ 当输入规模 $n \gg 2$ 后, 算法

需执行的基本操作次数 $T(n) = ?$

❖ 如欲更为精确地估计, 还可考查

需占用的存储单元数 $S(n) = ?$



Big-O notation

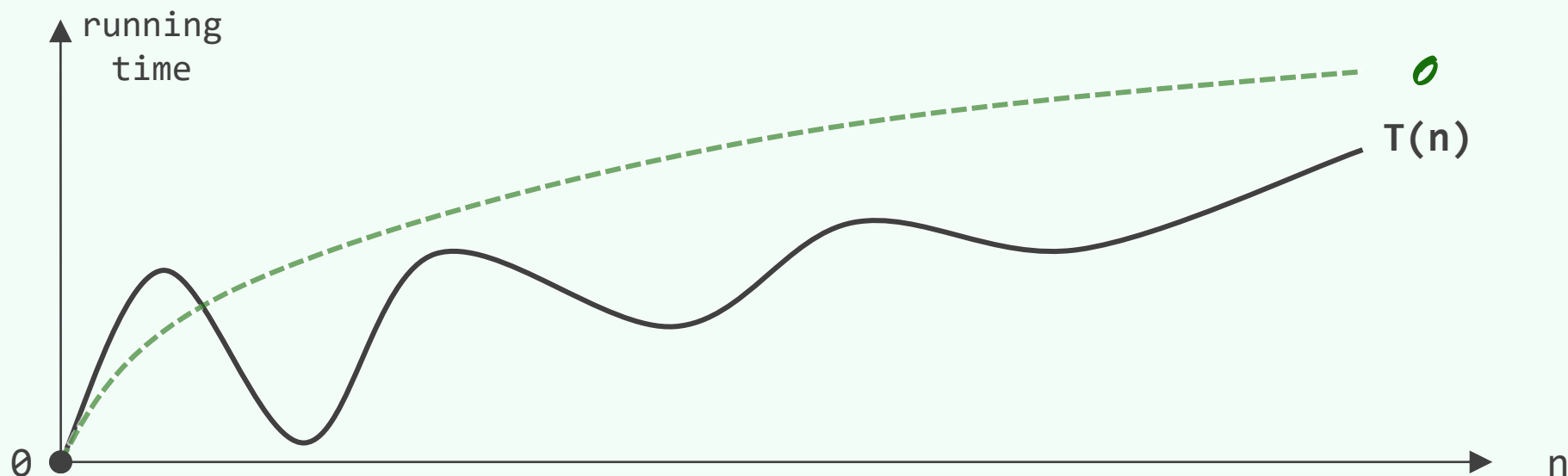
❖ **Paul Bachmann, 1894:** $T(n) = \mathcal{O}(f(n))$ **iff** $\exists c > 0$ **s.t.** $T(n) \leq c \cdot f(n) \quad \forall n \gg 2$

$$Ex: \sqrt{5n \cdot [3n \cdot (n+2) + 4] + 6} < \sqrt{5n \cdot [6n^2 + 4] + 6} < \sqrt{35n^3 + 6} < 6 \cdot n^{1.5} = \mathcal{O}(n^{1.5})$$

❖ 与 $T(n)$ 相比, $f(n)$ 在形式上更为简洁, 但依然反映前者的增长趋势

$$\mathcal{O}(f(n)) = \mathcal{O}(c \cdot f(n))$$

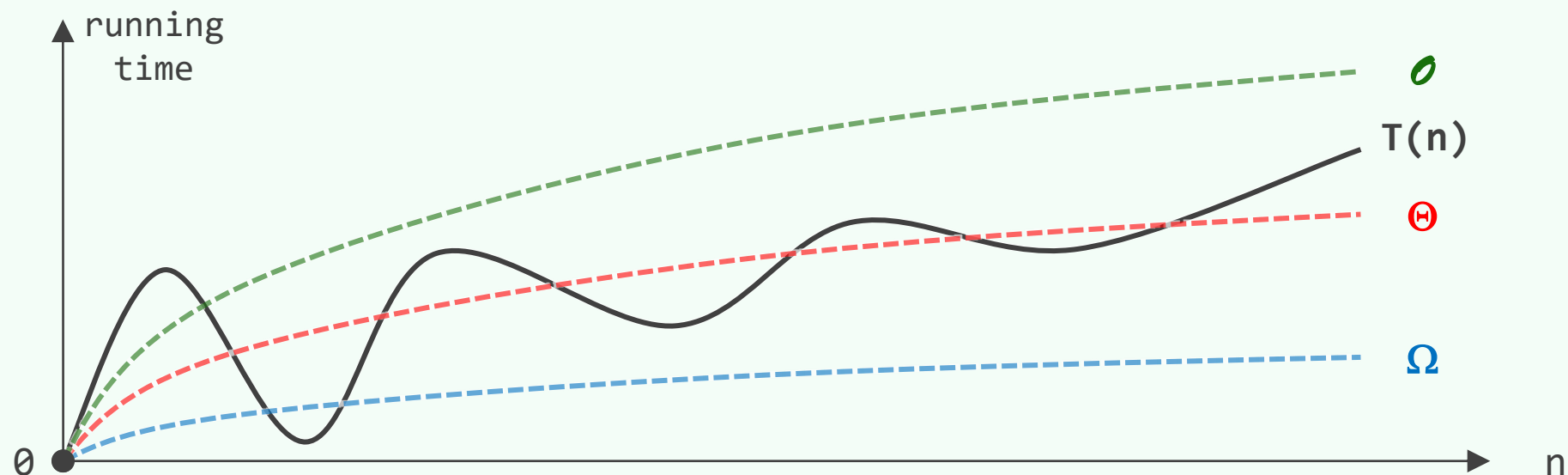
$$\mathcal{O}(n^a \pm n^b) = \mathcal{O}(n^a), \quad a \geq b > 0$$



其它记号

❖ $T(n) = \Omega(f(n))$ **iff** $\exists c > 0$ **s.t.** $T(n) \geq c \cdot f(n) \quad \forall n \gg 2$

❖ $T(n) = \Theta(f(n))$ **iff** $\exists c_1 \geq c_2 > 0$ **s.t.** $c_1 \cdot f(n) \geq T(n) \geq c_2 \cdot f(n) \quad \forall n \gg 2$



空间复杂度

❖ Storage Complexity:

算法执行过程中，会消耗多少空间

❖ 好消息：各种渐近记号，可以直接借用

❖ 约定：输入本身所占空间，通常不计
因为：属于固有消耗，与算法优劣无关

❖ 其余的所有，都需计入：
转储、缓冲、中继、索引、变换...

❖ 往往：不必重点考查空间复杂度

因为：无论是在TM还是RAM模型中

- 一次基本操作至多消耗 $O(1)$ 空间
- 时间复杂度就是空间复杂度天然的一个上界

❖ 如何才能达到紧界？

- TM: Head一旦出发，便义无反顾、誓不回头
- RAM: 每次都须开辟处女地，绝不故地重游

❖ 当然，若非常在乎空间效率，则需另当别论

绪论

渐近复杂度：多项式与可解

$\Theta(1) - O(2)$

Computational problems can be feasibly computed on some computational device only if they can be computed in polynomial time. - A. Cobham & J. Edmonds

邓俊辉

deng@tsinghua.edu.cn

$O(1)$: constant

❖ 常数: $2 = 2025 = 2025 \times 2025 = O(1)$

//含RAM的所有基本操作, 甚至

$$2025^{2025} = O(1)$$

❖ 从渐近的角度来看, 再大的常数, 也要小于递增的变数

//尽管实际并非如此

❖ [General Twin Prime Conjecture, de Polignac 1849]

For every natural number k , there are infinitely many prime pairs p and q
such that $p - q = 2k$

❖ [Yitang Zhang, April 2013] $k \leq 35,000,000$

[Terence Tao, May 2013] $k \leq 6,500,000$

[Polymath Project, April 2014] $k \leq 123$

...

$O(1)$: constant

❖ 这类算法的效率，自然已属最高了

//总不能奢望不劳而获吧

❖ 一定不含循环？

```
for ( i = 0; i < n; i += n/2025 + 1 );
```

//常数

```
for ( i = 1; i < n; i = 1 << i );
```

//log*n, 几乎常数

❖ 一定不含分支转向？

```
if ( (n + m) * (n + m) < 4 * n * m ) goto UNREACHABLE;
```

//不考虑溢出

❖ 一定不能有（递归）调用？

```
if ( 2 == (n * n) % 5 ) neverCalled(n);
```

//Quadratic Residue

❖ **空间**复杂度必然也是 $O(1)$ ：仅需常数辅助空间的算法，称作**就地算法**|In-place algorithm

$\mathcal{O}(\log^c n)$: poly-log

❖ 对数 $\mathcal{O}(\log n)$: $\ln n$ $\lg n$ $\log_{100} n$ $\log_{2025} n$ // 为何不注明底数?

❖ 常底数无所谓: $\forall a, b > 1, \log_a n = \boxed{\log_a b} \cdot \log_b n = \Theta(\log_b n)$

❖ 常数次幂无所谓: $\forall c > 0, \log n^c = c \cdot \log n = \Theta(\log n)$

❖ 对数多项式: $123 \cdot \log^{321} (n^{2025}) + 2025 \cdot \log^{123} (n^{321}) = \mathcal{O}(\log^{321} n)$

❖ 这类算法非常有效, 复杂度无限接近于常数: $\forall c > 0, \log n = \mathcal{O}(n^c)$

$\mathcal{O}(n^c)$: polynomial

❖ **多项式**: $a_k \cdot n^k + a_{k-1} \cdot n^{k-1} + \dots + a_2 \cdot n^2 + a_1 \cdot n + a_0 = \mathcal{O}(n^k), a_k > 0$

$$100 \cdot n + 2025 = \mathcal{O}(n)$$

$$\sqrt{23 \cdot n - 472} \times \sqrt{101 \cdot n + 2025} = \mathcal{O}(n)$$

$$(100 \cdot n - 532) \cdot (20 \cdot n^2 - 445 \cdot n + 2025) = \mathcal{O}(n^3) \quad (2025 \cdot n^2 - 129)/(1911 \cdot n - 37) = \mathcal{O}(n)$$

$$\sqrt[5]{4 \cdot n^4 \cdot \log^4 n + \sqrt[3]{43 \cdot n^{34} \cdot \log^{43} n + \sqrt[3]{432 \cdot n^{234} \cdot \log^{432} n + \sqrt[3]{4321 \cdot n^{1234} \cdot \log^{4321} n + n^{2025}}}}}} = \mathcal{O}(n^{15})$$

❖ **线性 (linear function)** : 所有 $\mathcal{O}(n)$ 类函数

❖ 从 $\mathcal{O}(n)$ 到 $\mathcal{O}(n^2)$, 是课程主要覆盖的范围: **低**了太稀有, 多以成为经典; **高**了, 基本上没法实用

❖ 这类算法的效率通常认为**已可**令人满意, 然而...这个标准是否**太低**了?

//P难度!

绪论

渐近复杂度：指数与难解

$\Theta(1) - O(3)$

慌得那拿盤的小怪，戰兢兢跑去報道：“難，難，難！難，難，難！”
老妖道：“怎麼有許多難？”

“你是什么东西？”太太说。四虎子也楞住了，他自己
不知道他是什么东西——这本是世上最难答的一个问题

邓俊辉

deng@tsinghua.edu.cn

$\mathcal{O}(2^n)$: exponential

❖ 指数: $T(n) = \mathcal{O}(a^n)$, $a > 1$

$$\because e^n = 1 + n + n^2/2! + n^3/3! + n^4/4! + \dots$$

$$\therefore \forall c > 0, a > 1, n^c = \mathcal{O}(a^n)$$

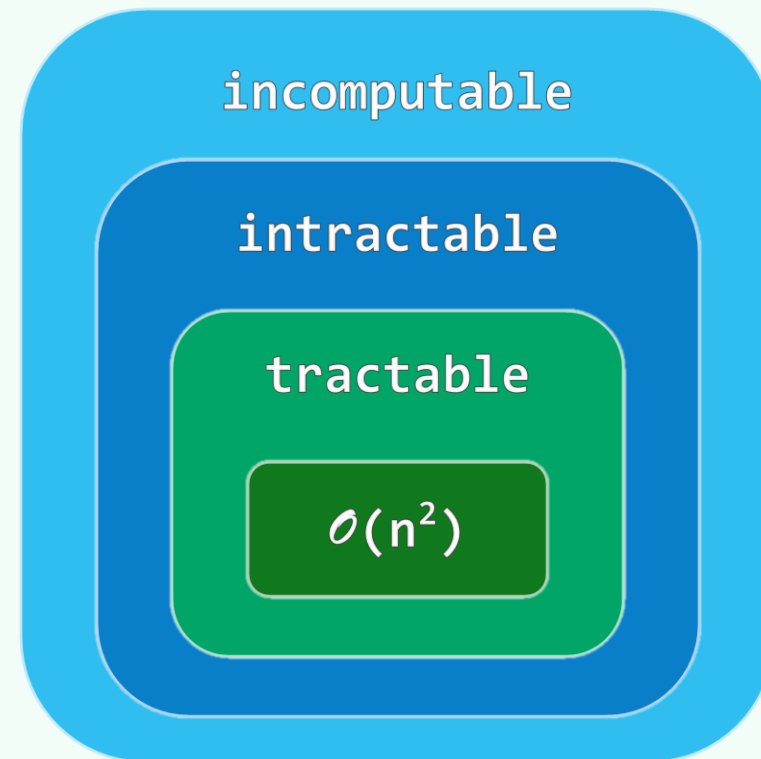
❖ 比如: $n^{2025} = \mathcal{O}(1.0001^n)$

❖ 这类算法的计算成本增长极快, 通常被认为**不可忍受**

❖ 从 $\mathcal{O}(n^c)$ 到 $\mathcal{O}(2^n)$, 是从**有效**算法到**无效**算法的分水岭

❖ $\mathcal{O}(2^n)$ 算法往往**显而易见**, 然而设计出 $\mathcal{O}(n^c)$ 算法却**极其不易**, 有时甚至注定是**徒劳无功**

❖ 更糟糕的是, 这类问题要远比我们想象的**多得多**...



SubsetSum: 问题

$$\forall S = \{ a_1, a_2, \dots, a_n \} \subset \mathbb{Z}^+$$

$$0 \leq t \leq s = \sum_{k=1}^n a_k$$

$$\exists T \subseteq S \text{ s.t. } \sum_{a \in T} a = t ?$$

❖ 从那堆石头里，曹冲真能挑出几块 //s

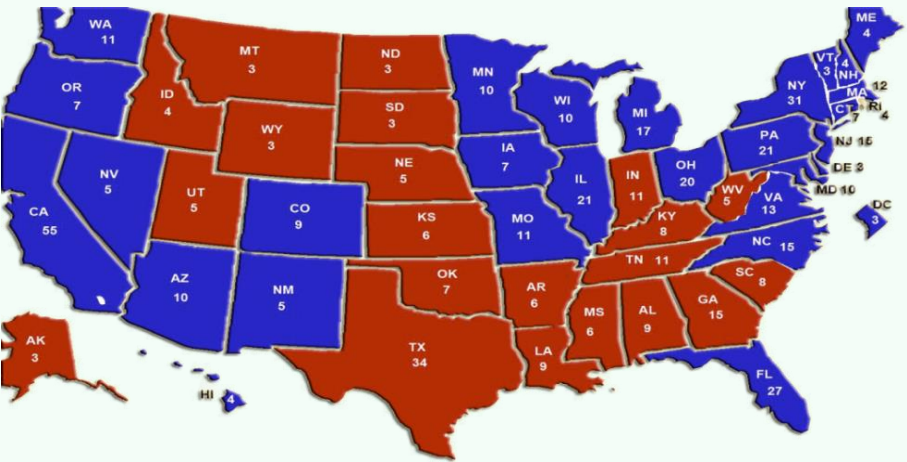
恰好与大象一样重? //t

❖ 选举人团投票制：51个选区，共538票 //n, 2t

若仅两位候选人，会否恰好各得269票? //t

可视作SubsetSum的特例： $s = \sum_{k=1}^n a_k = 2t$

55	California	11	Indiana	7	Connecticut	4	Idaho
34	Texas	11	Missouri	7	Iowa	4	Maine
31	New York	11	Tennessee	7	Oklahoma	4	New Hampshire
27	Florida	11	Washington	7	Oregon	4	Rhode Island
21	Illinois	10	Arizona	6	Arkansas	3	Alaska
21	Pennsylvania	10	Maryland	6	Kansas	3	Delaware
20	Ohio	10	Minnesota	6	Mississippi	3	D. C.
17	Michigan	10	Wisconsin	5	Nebraska	3	Montana
15	Georgia	9	Alabama	5	Nevada	3	North Dakota
15	New Jersey	9	Colorado	5	New Mexico	3	South Dakota
15	North Carolina	9	Louisiana	5	Utah	3	Vermont
13	Virginia	8	Kentucky	5	West Virginia	3	Wyoming
12	Massachusetts	8	South Carolina	4	Hawaii	538 = Σ	



SubsetSum: ~~算法~~ 程序

❖ 直觉上, 似乎并**不难**:

逐一**枚举** S 的每一子集, 分别统计总和并核对

❖ $sSum(S = \{a_1, a_2, \dots, a_n\}, t)$

if ($t == 0$) return true;

if ($n == 0$) return false;

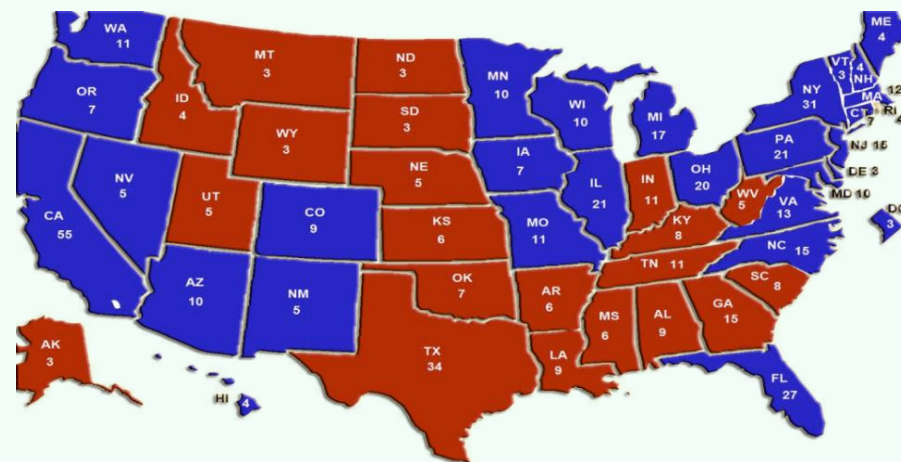
$S = S \setminus \{a_n\}$; //classification

return $sSum(S, t - a_n) \parallel sSum(S, t)$;

❖ 最坏情况下, 总需检视每一个子集, 然而...

$$|2^S| = 2^{|S|} = 2^n$$

55	California	11	Indiana	7	Connecticut	4	Idaho
34	Texas	11	Missouri	7	Iowa	4	Maine
31	New York	11	Tennessee	7	Oklahoma	4	New Hampshire
27	Florida	11	Washington	7	Oregon	4	Rhode Island
21	Illinois	10	Arizona	6	Arkansas	3	Alaska
21	Pennsylvania	10	Maryland	6	Kansas	3	Delaware
20	Ohio	10	Minnesota	6	Mississippi	3	D. C.
17	Michigan	10	Wisconsin	5	Nebraska	3	Montana
15	Georgia	9	Alabama	5	Nevada	3	North Dakota
15	New Jersey	9	Colorado	5	New Mexico	3	South Dakota
15	North Carolina	9	Louisiana	5	Utah	3	Vermont
13	Virginia	8	Kentucky	5	West Virginia	3	Wyoming
12	Massachusetts	8	South Carolina	4	Hawaii	538 = Σ	



SubsetSum: NPC

❖ 故严格地讲

这类方法只能算作程序，而非算法

❖ 还是直觉：应该有更好的办法吧？

比如...转化为...背包问题...?

很遗憾，这里对于整数的取值范围未作任何假定

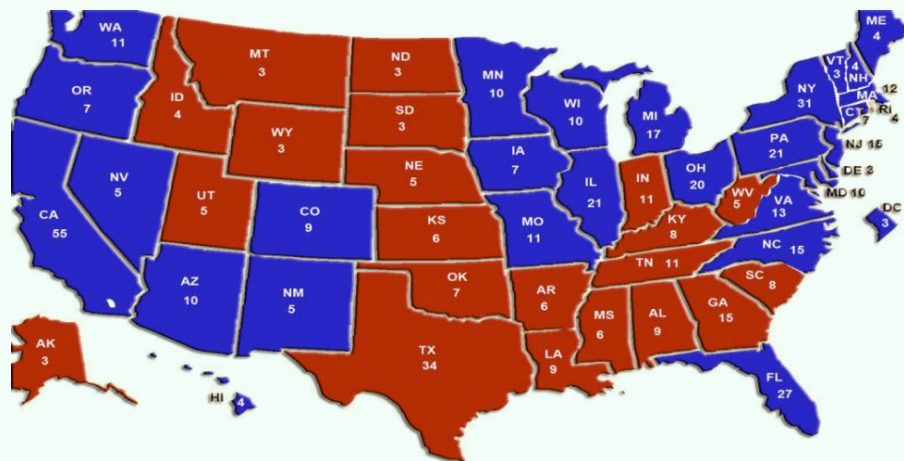
❖ SubsetSum is NP-complete —— 什么意思?

❖ 意即：就目前的计算模型而言

不存在可在多项式时间内解决此问题的算法

上述的直觉算法，居然就是最优的

55	California	11	Indiana	7	Connecticut	4	Idaho
34	Texas	11	Missouri	7	Iowa	4	Maine
31	New York	11	Tennessee	7	Oklahoma	4	New Hampshire
27	Florida	11	Washington	7	Oregon	4	Rhode Island
21	Illinois	10	Arizona	6	Arkansas	3	Alaska
21	Pennsylvania	10	Maryland	6	Kansas	3	Delaware
20	Ohio	10	Minnesota	6	Mississippi	3	D. C.
17	Michigan	10	Wisconsin	5	Nebraska	3	Montana
15	Georgia	9	Alabama	5	Nevada	3	North Dakota
15	New Jersey	9	Colorado	5	New Mexico	3	South Dakota
15	North Carolina	9	Louisiana	5	Utah	3	Vermont
13	Virginia	8	Kentucky	5	West Virginia	3	Wyoming
12	Massachusetts	8	South Carolina	4	Hawaii	538 = Σ	



渐近复杂度层次

常数	$O(1)$	再好不过，但难得如此幸运	对数据结构的基本操作
	$O(\log^*n)$	在这个宇宙中，几乎就是常数	逆Ackermann函数
对数	$O(\log n)$	与常数无限接近，且不难遇到	有序向量的二分查找；堆、词典的查询、插入与删除
线性	$O(n)$	努力目标，经常遇到	树、图的遍历
	$O(n \log^*n)$	几乎几乎几乎...就是线性	某些MST算法
	$O(n \log \log n)$	非常非常非常...接近线性	某些三角剖分算法
	$O(n \log n)$	最常出现，但不见得最优	排序、EU、Huffman编码
平方	$O(n^2)$	所有输入对象两两组合	Dijkstra算法
立方	$O(n^3)$	不常见	矩阵乘法
多项式	$O(n^c)$	P问题 = 存在多项式算法的问题	
指数	$O(2^n)$	很多问题的平凡算法，再尽可能优化	
...		绝大多数问题，并不存在算法	

绪论

复杂度分析：封底估算

He calculated just as men breathe, as eagles
sustain themselves in the air.

我仿佛是横越三世来见你的，而你却不在

谁校对时间，谁就会突然老去

邓俊辉

deng@tsinghua.edu.cn

Back-Of-The-Envelope Calculation

❖ 赤道周长 $\approx 787 \times 360/7.2 = 787 \times 50 = 39,350$ km

❖ 1天 = 24小时 $\times 60$ 分 $\times 60$ 秒 $\approx 25 \times 4000 = 10^5$ 秒

1世 = 1代 = 30年 $\times 365$ 天 = 10^4 天 = 10^9 秒

1生 $\approx$ 1世纪 = 100年 $\times 365$ 天 = 3×10^4 天 = 3×10^9 秒

三生三世 $\approx$ 300年 = 10^{10} 秒

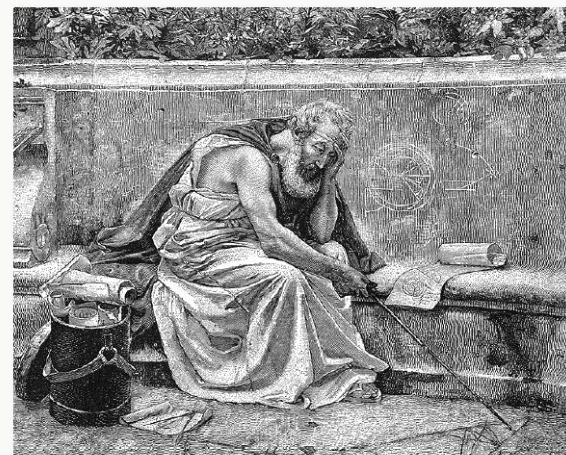
❖ 为祖国健康工作五十年 = 1,600,000,000秒

宇宙大爆炸至今 = 4×10^{17} 秒

1 googol = 10^{100}



Enrico Fermi
(1901 - 1954)



Eratosthenes
(276 ~ 194 B.C.)

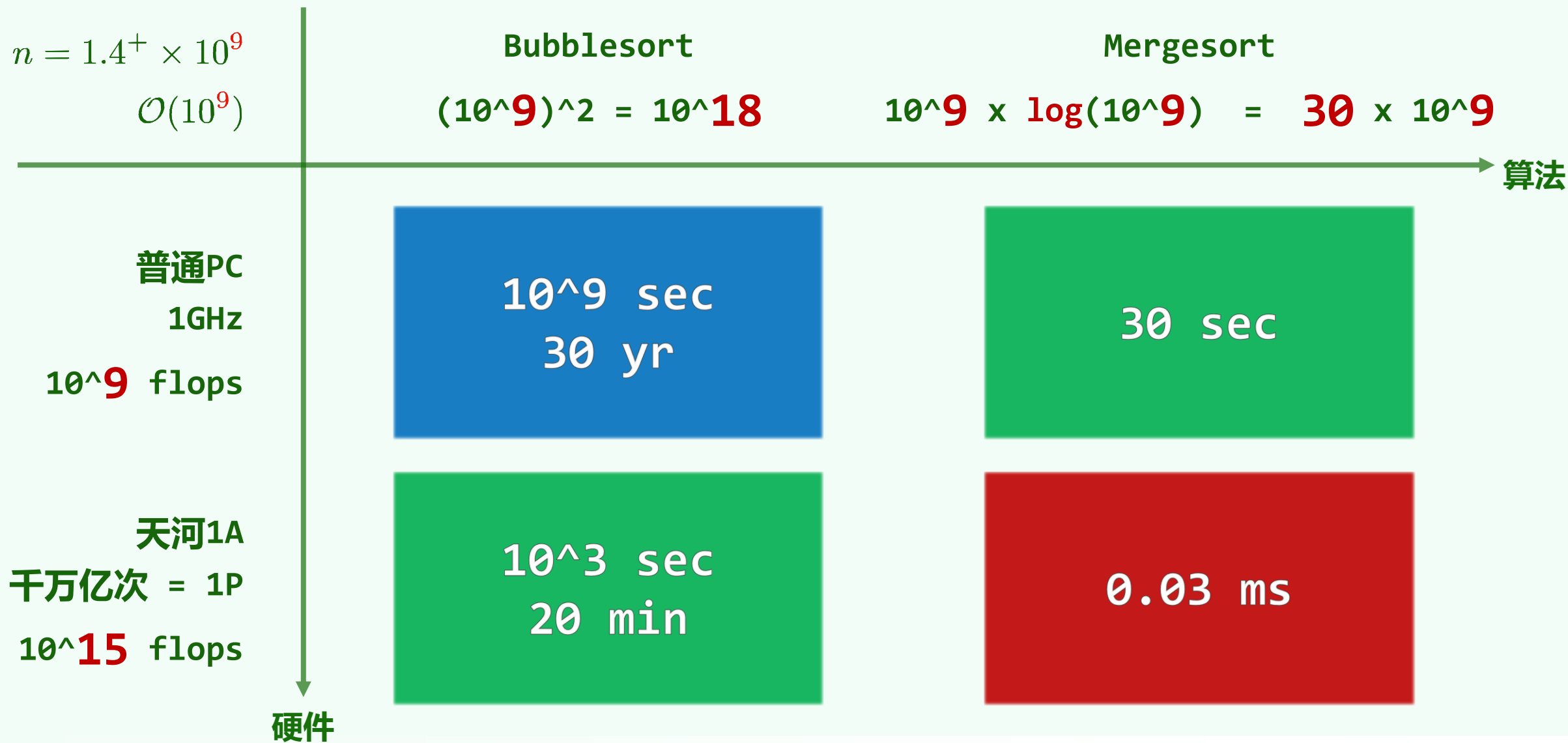
渐近时间复杂度

n	10^3	10^6	10^9	10^{12}	10^{15}	10^{18}	10^{21}
$\mathcal{O}(\log^* n)$	3	≤ 4 (if $n < 2^{(2^{(2^{(2^2)})})} = 10^{19660}$)					
$\mathcal{O}(\log \log n)$	3	< 10 (if $n < 10^{300}$)					
$\mathcal{O}(\log n)$	10	20	30	40	50	60	70
$\mathcal{O}(\sqrt{n})$	30	10^3	$3 \cdot 10^4$	10^6	$3 \cdot 10^7$	10^9	$3 \cdot 10^{10}$
$\mathcal{O}(n)$	10^3	10^6	10^9	10^{12}	10^{15}	10^{18}	10^{21}
$\mathcal{O}(n \cdot \log n)$	10^4	$2 \cdot 10^7$	$3 \cdot 10^{10}$	$4 \cdot 10^{13}$	$5 \cdot 10^{16}$	$6 \cdot 10^{19}$	$7 \cdot 10^{22}$
$\mathcal{O}(n \cdot \sqrt{n})$	$3 \cdot 10^4$	10^9	$3 \cdot 10^{13}$	10^{18}	$3 \cdot 10^{22}$	10^{27}	$3 \cdot 10^{31}$
$\mathcal{O}(n^2)$	10^6	10^{12}	10^{18}	10^{24}	10^{30}	10^{36}	10^{42}
$\mathcal{O}(n^3)$	10^9	10^{18}	10^{27}	10^{36}	10^{45}	10^{54}	10^{63}

假如你的电脑，每秒能够执行10^9次基本运算

n	10^3	10^6	10^9	10^12	10^15	10^18	10^21
$\mathcal{O}(\log^* n)$	几乎...就是 $\mathcal{O}(1)$						
$\mathcal{O}(\log \log n)$	非常非常非常...接近于 $\mathcal{O}(1)$						
$\mathcal{O}(\log n)$	< 0.1 us = 0.0001 ms, 毫无压力						
$\mathcal{O}(\sqrt{n})$		1 us	30 us	1 ms	30 ms	1 sec	30 sec
$\mathcal{O}(n)$	1 us	1 ms	1 sec	17 min	12 day	30 yr	300 c
$\mathcal{O}(n \cdot \log n)$	10 us	20 ms	30 sec	11 hr	579 day	18 century	21,000 c
$\mathcal{O}(n \cdot \sqrt{n})$	30 us	1 sec	8 hr	30 yr	10^5 c	3*10^8 c	10^13 c
$\mathcal{O}(n^2)$	1 ms	17 min	30 yr	3*10^5 c	3*10^11 c	3*10^17 c	3*10^23 c
$\mathcal{O}(n^3)$	1 sec	30 yr	3*10^8 c	3*10^17 c	3*10^26 c	3*10^35 c	3*10^44 c

实例：全国人口普查数据的排序



绪论

复杂度分析：可视化

All knowledge begins with sensory experience, and especially with sight.

- Leonardo. da Vinci

A good graphic is a microscope to see patterns in numbers.

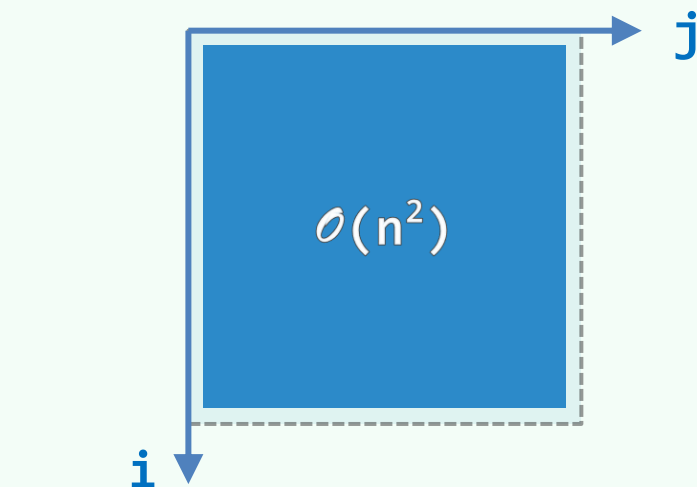
- Edward Tufte

邓俊辉

deng@tsinghua.edu.cn

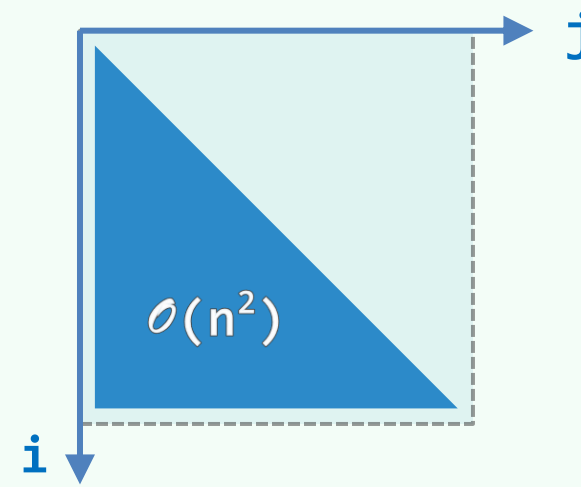
迭代 + 算术级数

```
for( int i = 0; i < n; i++ )  
    for( int j = 0; j < n; j++ )  
        O1op(const i, const j);
```



$$\sum_{i=0}^{n-1} n = n \times n = O(n^2)$$

```
for( int i = 0; i < n; i++ )  
    for( int j = 0; j < i; j++ )  
        O1op(const i, const j);
```



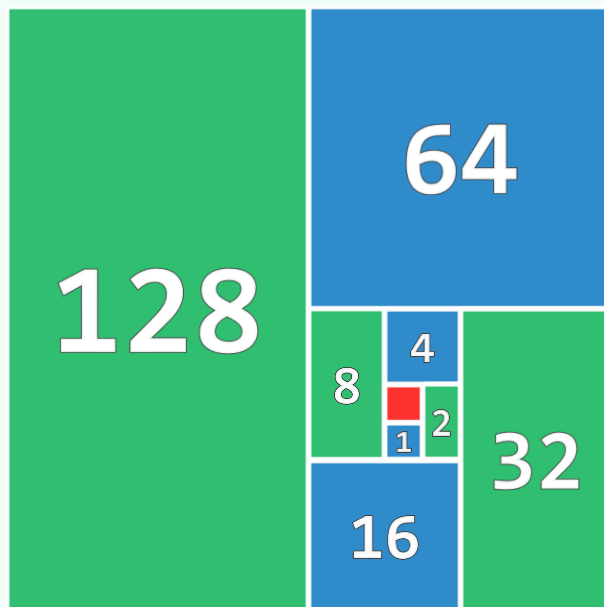
$$\sum_{i=0}^{n-1} i = n \cdot (n - 1) / 2 = O(n^2)$$

迭代 vs. 级数

```
for( int i = 1; i < n; i <<= 1 )
```

```
    for( int j = 0; j < i; j++ )
```

```
        O1op( const i, const j );
```



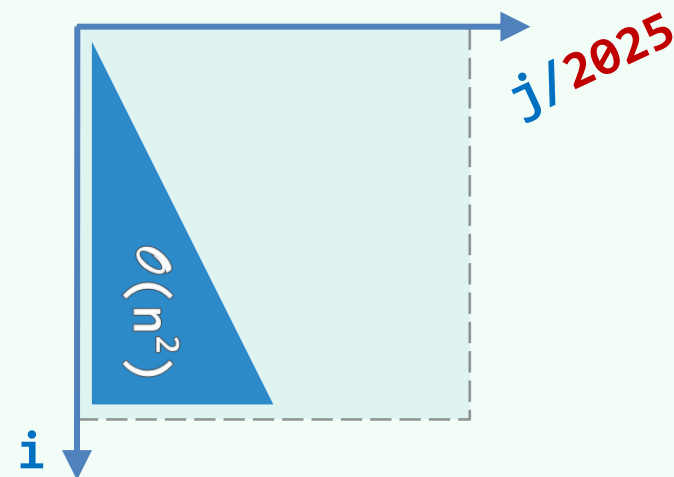
$$1 + 2 + 4 + \dots + 2^{\lfloor \log_2 (n-1) \rfloor}$$

$$= 2^{\lceil \log_2 n \rceil} - 1 = \mathcal{O}(n)$$

```
for( int i = 0; i < n; i++ )
```

```
    for( int j = 0; j < i; j += 2025 )
```

```
        O1op( const i, const j );
```



迭代 + 复杂级数

```
for( int i = 0; i <= n; i++ )  
    for( int j = 1; j < i; j += j )  
        O1op( const i, const j );
```

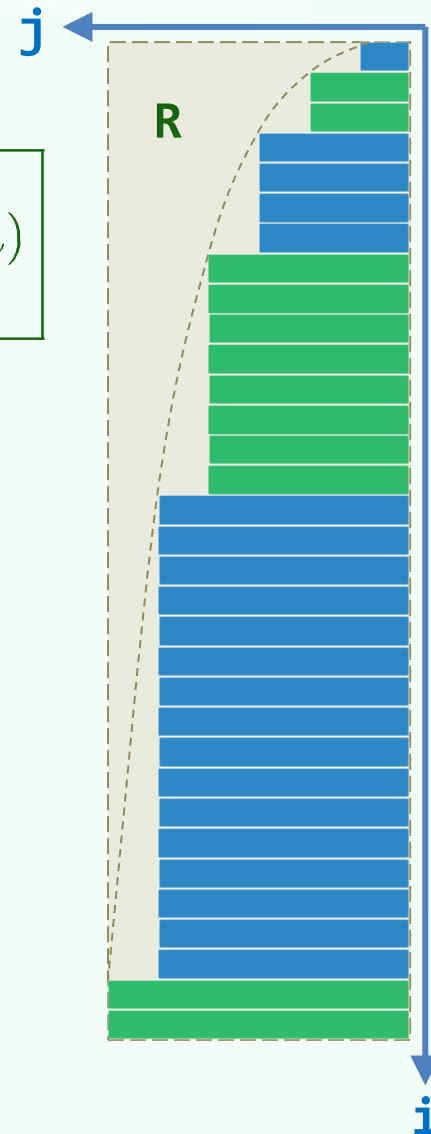
$$T(n) = \sum_{i=0}^n \lceil \log_2 i \rceil = \mathcal{O}(n \log n)$$

$$T(n) \approx \int_{x=0}^n \ln(x) dx = n \cdot \ln(n) - \int_{x=0}^n x \cdot d(\ln(x)) = n \cdot \ln(n) - n$$

$$T(n) = \sum_{k=1}^{\log n} k \cdot 2^{k-1} = \sum_{k=1}^{\log n} \sum_{i=1}^k 2^{k-1} = \sum_{i=1}^{\log n} \sum_{k=i}^{\log n} 2^{k-1}$$

$$T(n) \leq \sum_{i=1}^{\log n} \sum_{k=1}^{\log n} 2^{k-1} \leq \sum_{i=1}^{\log n} 2^{\log n} = \sum_{i=1}^{\log n} n = n \cdot \log n$$

$$T(n) \geq \sum_{i=1}^{\log n} \sum_{k=\log n}^{\log n} 2^{k-1} \geq \sum_{i=1}^{\log n} 2^{\log n-1} = \sum_{i=1}^{\log n} n/2 = n/2 \cdot \log n$$



绪论

复杂度分析：级数

01-D3

Go To Statement Considered Harmful. - E. Dijkstra, 1968

主啊，我向你承认，我依旧不明白时间是什么

邓俊辉

deng@tsinghua.edu.cn

从跳转，到循环

❖ 没有**分支转向**的代码是没有**灵魂**的，不过是一杯**白开水**

- 函数调用（含递归）本质上都是GoTo
- 不加约束的GoTo：太任性，会毒害代码！

❖ **结构化编程**：将Goto关进笼子，**隐藏**起来|不是**消灭**

规范的循环：for、while/do、repeat/until、... //本质上就是 “If + GoTo”

```
for ( int i = 0, j = 0; (i < n) && (j < m); i++, j <= 1 )
```

initialization termination update

❖ 规范的循环，其执行过程也必然会呈现出某种...简明的**规律**

为此，需要首先在**数学**上做些热身...

级数

❖ **算术级数：与末项平方同阶** $\sum_{k=1}^n k = 1 + 2 + \dots + n = \cancel{\frac{(n+1)}{2}} = \cancel{\frac{n(n+1)}{2}} = \mathcal{O}(n^2)$

❖ **幂方级数：比幂次高出一阶** $\sum_{k=0}^n k^d \approx \int_0^n x^d dx = \cancel{\frac{x^{d+1}}{d+1} \Big|_0^n} = \cancel{\frac{n^{d+1}}{d+1}} = \mathcal{O}(n^{d+1})$

$$\sum_{k=1}^n k^2 = 1^2 + 2^2 + 3^2 + \dots + n^2 = \cancel{n(n+1)(2n+1)/6} = \mathcal{O}(n^3)$$

$$\sum_{k=1}^n k^3 = 1^3 + 2^3 + 3^3 + \dots + n^3 = \cancel{n^2(n+1)^2/4} = \mathcal{O}(n^4)$$

$$\sum_{k=1}^n k^4 = 1^4 + 2^4 + 3^4 + \dots + n^4 = \cancel{n(n+1)(2n+1)(3n^2+3n-1)/30} = \mathcal{O}(n^5)$$

❖ **几何级数：与末项同阶**

$$\sum_{k=0}^n a^k = a^0 + a^1 + a^2 + a^3 + \dots + a^n = \cancel{\frac{a^{n+1}-1}{a-1}} = \mathcal{O}(a^n), \quad a > 1$$

$$\sum_{k=0}^n 2^k = 1 + 2 + 4 + 8 + \dots + 2^n = \cancel{2^{n+1}-1} = \cancel{\mathcal{O}(2^{n+1})} = \mathcal{O}(2^n)$$

收敛级数

$$\diamond \quad \sum_{k=2}^n \frac{1}{(k-1) \cdot k} = \frac{1}{1 \cdot 2} + \frac{1}{2 \cdot 3} + \frac{1}{3 \cdot 4} + \dots + \frac{1}{(n-1) \cdot n} = 1 - \frac{1}{n} = \mathcal{O}(1)$$

$$\sum_{k=1}^n \frac{1}{k^2} = 1 + \frac{1}{2^2} + \frac{1}{3^2} + \dots + \frac{1}{n^2} < 1 + \frac{1}{2^2} + \frac{1}{3^2} + \dots = \frac{\pi^2}{6} = \mathcal{O}(1)$$

$$\sum_{k \text{ is a perfect power}} \frac{1}{k-1} = \frac{1}{3} + \frac{1}{7} + \frac{1}{8} + \frac{1}{15} + \frac{1}{24} + \frac{1}{26} + \frac{1}{31} + \frac{1}{35} + \dots = 1 = \mathcal{O}(1)$$

$$\diamond \text{ 几何分布: } (1 + 2\lambda + 3\lambda^2 + 4\lambda^3 + \dots) \cdot (1 - \lambda)$$

$$= 1 + \lambda + \lambda^2 + \lambda^3 + \dots = 1/(1 - \lambda) = \mathcal{O}(1), \quad 0 < \lambda < 1$$

❖ 有必要讨论这类级数吗？难道，基本操作次数、存储单元数可能是分数？

是的，某种意义上的确是！

发散级数

❖ 调和级数: $\sum_{k=1}^n 1/k = 1 + 1/2 + 1/3 + 1/4 + \dots + 1/n = \ln n + \gamma + \mathcal{O}(\frac{1}{2n}) = \Theta(\log n)$

❖ 对数级数: $\sum_{k=1}^n \ln k = \ln \prod_{k=1}^n k = \ln n! \approx (n + 0.5) \cdot \ln n - n = \Theta(n \cdot \log n)$

❖ 对数 + 线性 + 指数: $\sum_{k=1}^n k \cdot \log k \approx \int_1^n x \ln x dx = x^2 \cdot (2 \cdot \ln x - 1)/4 \Big|_1^n = \mathcal{O}(n^2 \log n)$

$$\begin{aligned} \sum_{k=1}^n k \cdot 2^k &= \sum_{k=1}^n k \cdot 2^{k+1} - \sum_{k=1}^n k \cdot 2^k \\ &= (1/2) \cdot \left(\sum_{k=1}^{n+1} (k-1) \cdot 2^{k+1} - \sum_{k=1}^{n+1} (k-1) \cdot 2^k \right) = (n-1) \cdot 2^{n+1} + 2 = \mathcal{O}(n \cdot 2^n) \end{aligned}$$

❖ 如有兴趣, 不妨读读: [Concrete Mathematics](#)

//ex-2.35, Goldbach Theorem

绪论

迭代与递归：减而治之

让最后一个数结束这些争吵...

否则我就利用你留给我的权限

像一根一根扯马尾鬃一般

——删去个位数直至啥也看不见

您也会被我的推理逗着玩

雖我之死，有子存焉；子又生孫，孫又生子；子又有子，子又有孫；子子孫孫無窮匱也，而山不加增，何苦而不平？

邓俊辉

deng@tsinghua.edu.cn

Sum: 计算任意n个整数之和

```
❖ int SumI( int A[], int n ) {  
    int sum = 0; //累计器  
    for ( int i = 0; i < n; i++ )  
        sum += A[i]; //逐项累计  
    return sum; //返回累计值  
}
```

❖ 时间主要消耗于循环

因此: $T(n) = \mathcal{O}(n) = \Omega(n) = \Theta(n)$

且最好、最坏情况一样

❖ 空间呢?

你应该记得...

❖ ...空间复杂度是

- 除却**输入**数据, 只计入
- 在算法过程中所使用的**辅助**空间量

❖ 此处只需区区常数个辅助变量

累计不过 $\mathcal{O}(1)$ 空间

属于**就地**算法

Decrease-and-conquer

❖ 为求解一个大规模的问题

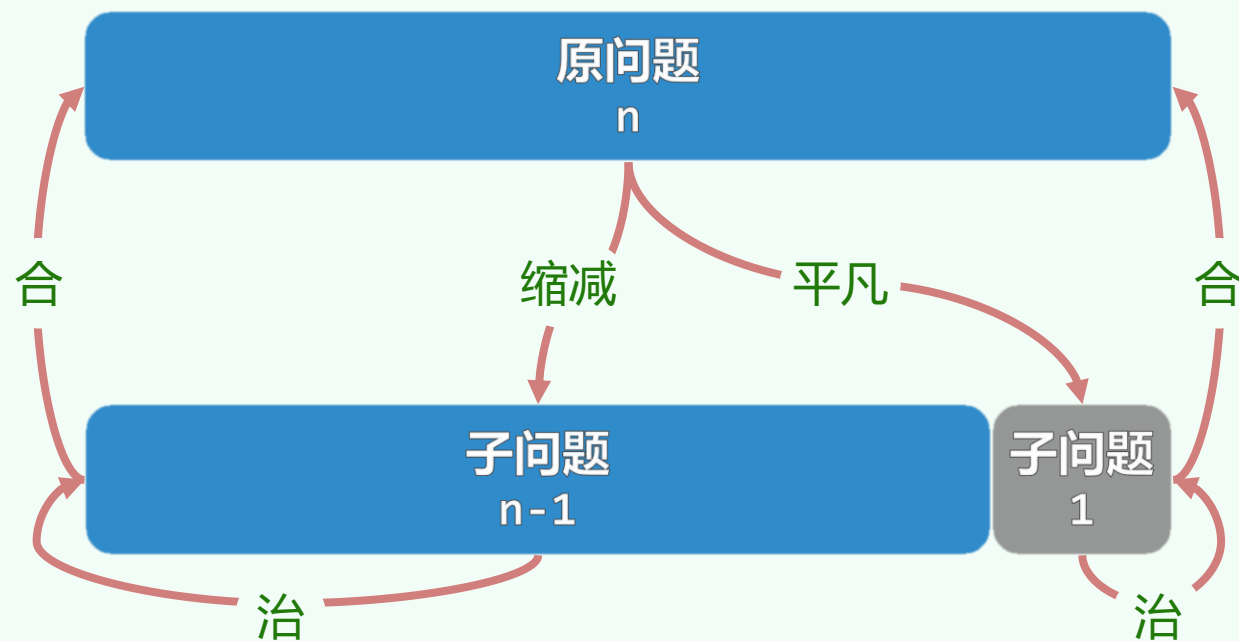
何不效仿愚公移山...

❖ 将其**分解**为两个子问题

- 子问题的形式不变
- 其一**平凡**，另一规模**缩减** //单调性

❖ **递归**求解非平凡的子问题

❖ **综合**子问题的解，得到原问题的解



Linear Recursion: Trace

```
int sum( int A[], int n )  
{ return n < 1 ? 0 : sum(A, n - 1) + A[n - 1]; }
```

❖ **递归跟踪**: 绘出**调用树** | call tree

计算过程中出现过的所有**调用实例**及其调用关系

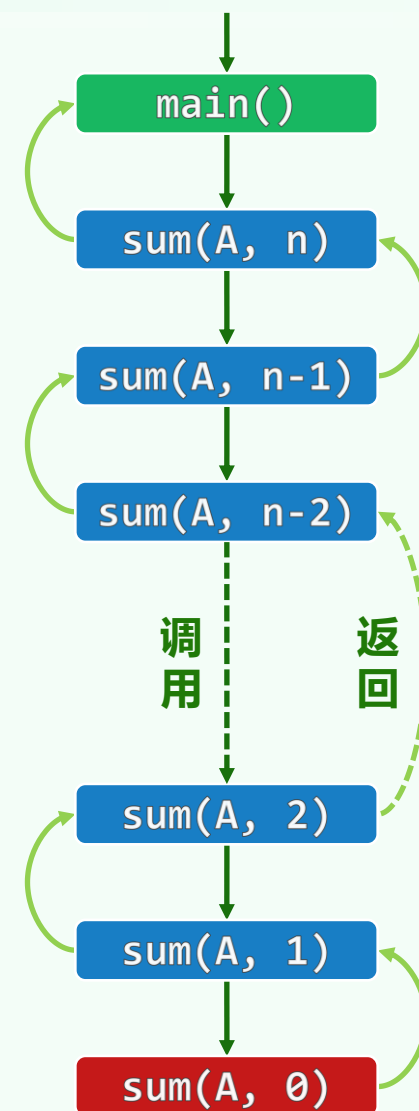
❖ 各实例所需时间之总和, 即为整体运行时间

调用操作本身的成本, 如何处理?

❖ 本例中, 共计 $n+1$ 个调用实例, 各自只需 $\mathcal{O}(1)$ 时间

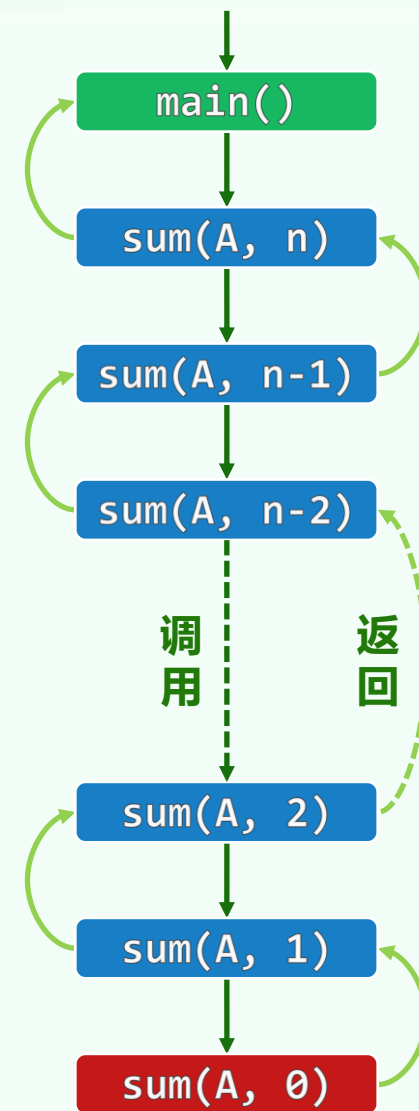
故总体运行时间为: $T(n) = \mathcal{O}(1) \times (n + 1) = \mathcal{O}(n)$

❖ 空间复杂度呢?



Linear Recursion: Recurrence

- ❖ 对于**大规模**的问题、**复杂**的递归算法，递归跟踪不再适用
此时可改用**抽象分析**的方法...
- ❖ 从**递推**的角度看，为求解规模为 n 的问题 $\text{sum}(A, n)$ ，需 $//T(n)$
 - **递归**地求解规模为 $n-1$ 的问题 $\text{sum}(A, n-1)$ ，再 $//T(n-1)$
 - 累加上 $A[n-1]$ $//O(1)$
- ❖ **递推方程**: $T(n) = T(n-1) + O(1)$ $//\text{recurrence}$
 $T(0) \equiv O(1)$ $//\text{base: } \text{sum}(A, 0)$
- ❖ **求解**: $T(n) = T(n-2) + O(2) = T(n-3) + O(3) = \dots$
 $= T(0) + O(n) = O(n)$



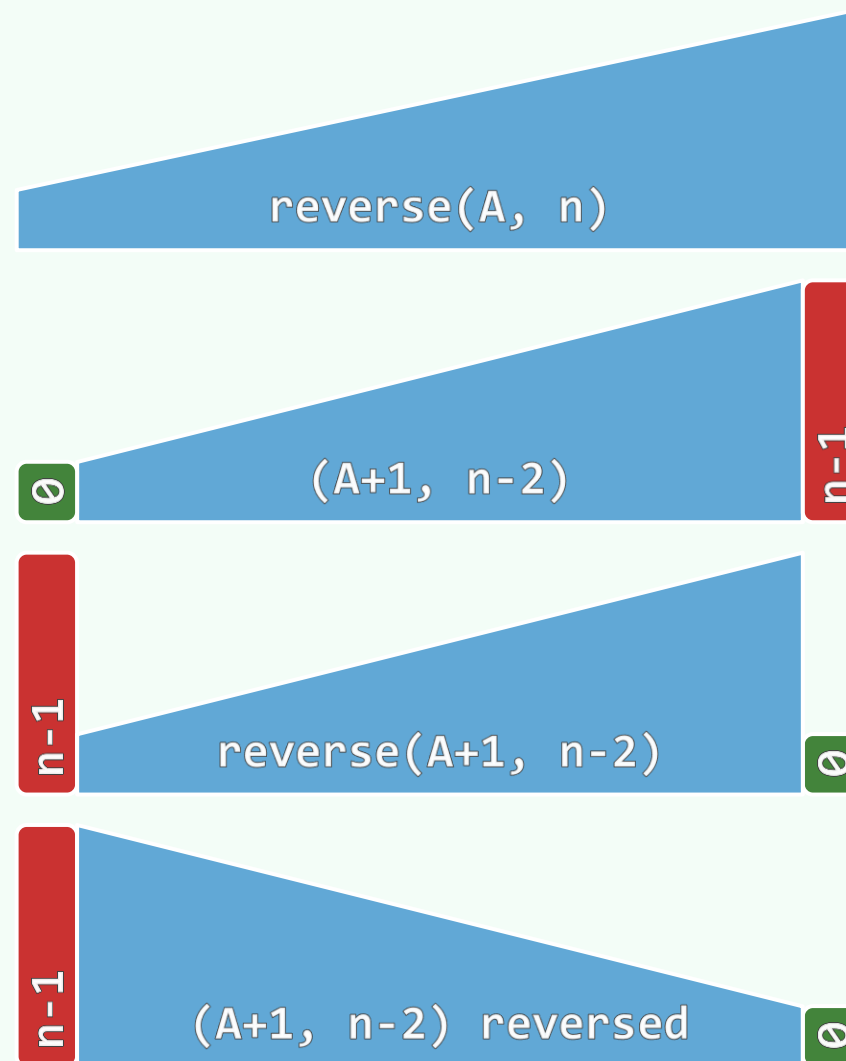
Reverse: 数组倒置

❖ 减而治之

```
❖ void reverse( int * A, int n ) {  
    if ( n < 2 ) return;  
    swap( A[0], A[n - 1] );  
    reverse( A + 1, n - 2 );  
}
```

❖ 线性递归: $O(n)$

❖ 稍后: 尾递归 ~ 迭代化



绪论

迭代与递归：分而治之

01-E2

邓俊辉

deng@tsinghua.edu.cn

凡治眾如治寡，分數是也

Divide-and-Conquer

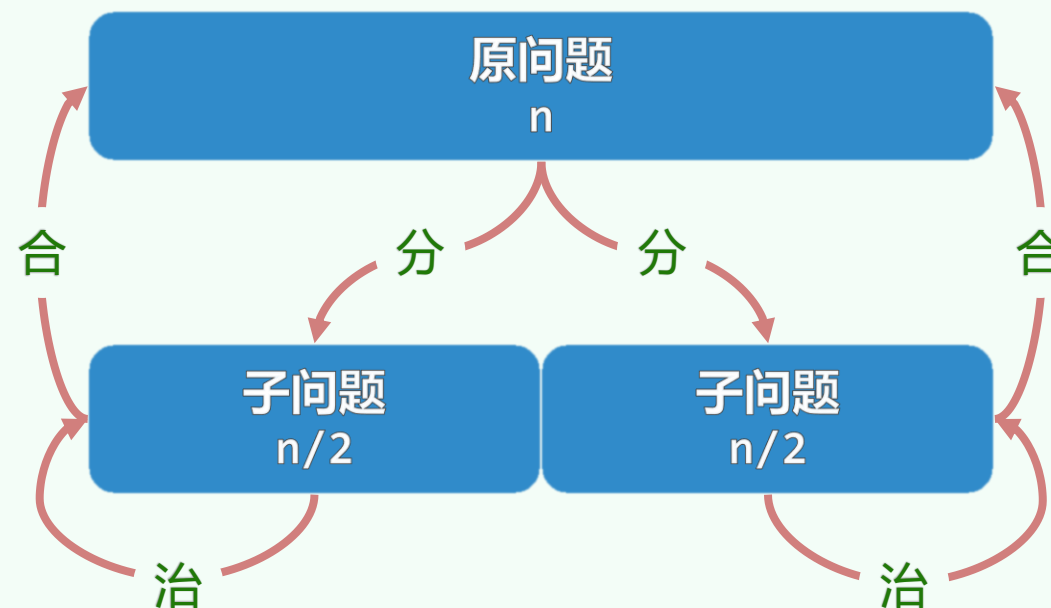
❖ 为求解一个大规模的问题，可以...

❖ 将其**划分**为若干子问题

- 通常两个，且规模**大体相当**
- 子问题的形式不变

❖ 分别**递归**求解各个子问题

❖ **综合**子问题的解，得到原问题的解



Binary Recursion

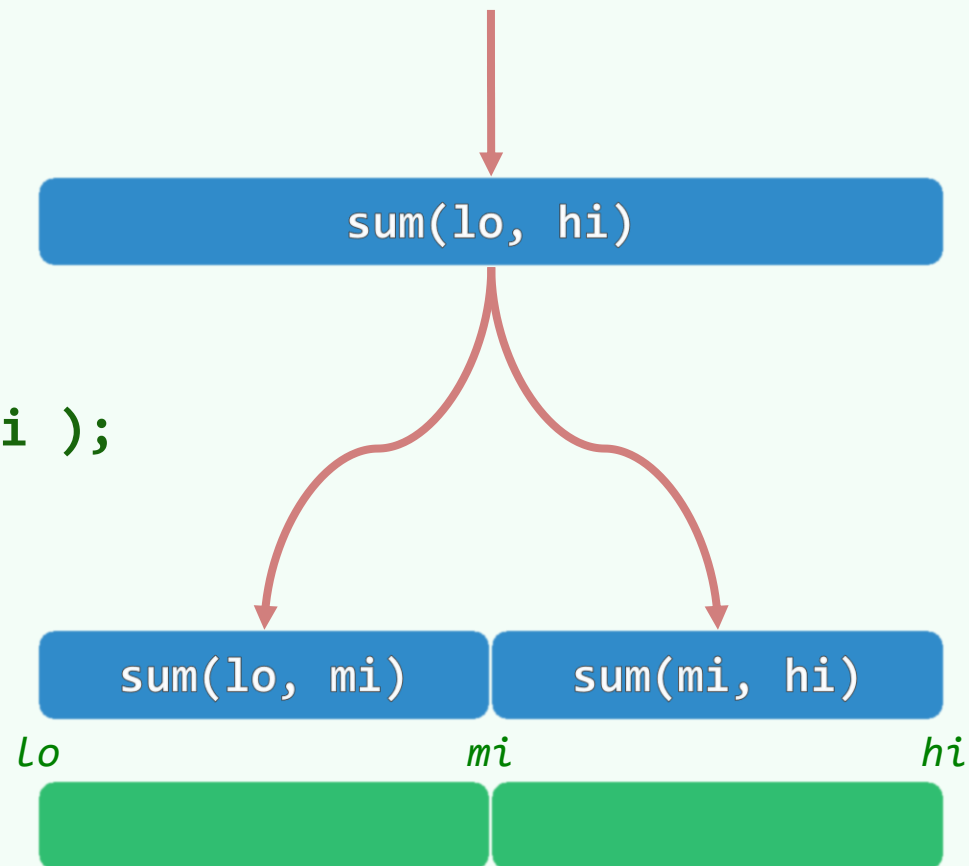
```
❖ int sum( int A[], int lo, int hi ) {  
    if ( hi - lo < 2 ) return A[lo];  
    int mi = (lo + hi) >> 1; //切分  
    return sum( A, lo, mi ) + sum( A, mi, hi );  
}
```

❖ **左闭右开**的惯例:

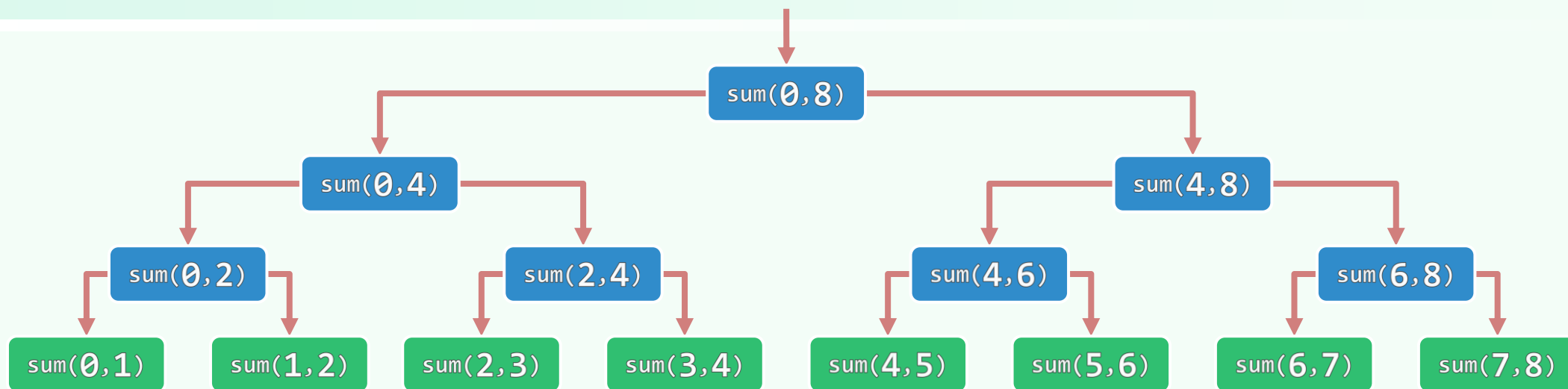
指定区间范围为 $A[lo, hi)$

❖ 整体求和: $sum(A, 0, n)$

❖ 原理及正确性: 加法满足**结合律**, 最终结果与执行次序无关



Binary Recursion: Trace



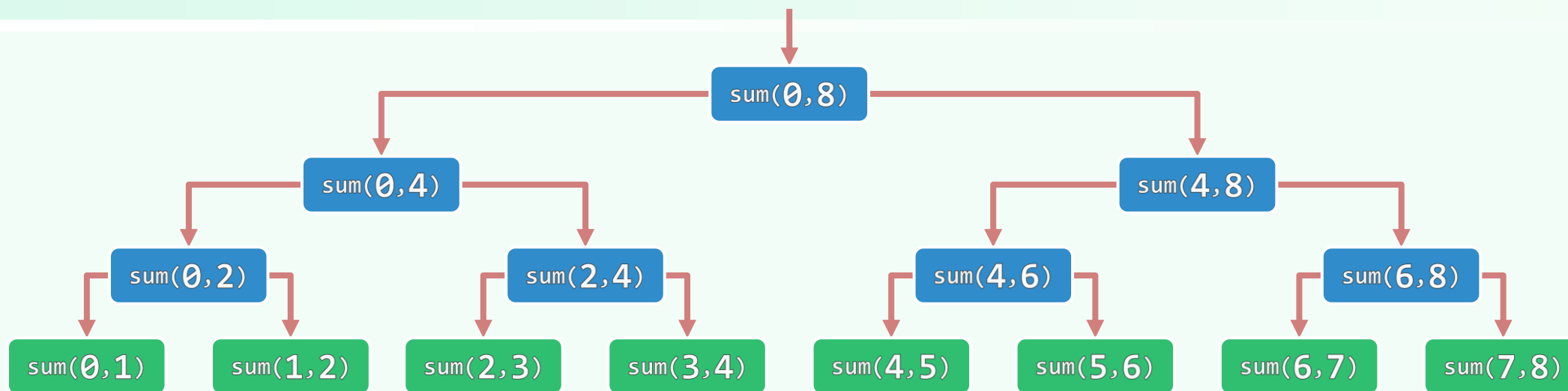
❖ 观察：每个递归实例仅需 $O(1)$ 时间 --> 运行时间决定于**所有**递归实例的总数

$$T(n) = O(1) \times (2^0 + 2^1 + 2^2 + \dots + 2^{\log n}) = \dots = O(n)$$

观察：分层统计，递归实例数量呈**几何级数** --> 运行时间决定于**底层**递归实例的总数

❖ 每个元素，恰好**各自**对应于一个**递归基**...

Binary Recursion: Storage



❖ 万变不离其宗，空间复杂度取决于

空间占用量**最大**的时刻

❖ 递归算法的运行 = Theseus遍历迷宫

空间占用量 = 绳线长度

❖ 在每一递归基，绳线长度都是**极大**

❖ 所有极大中的最大者，称作递归**深度**

即是空间复杂度

就此例而言，等于 $\mathcal{O}(\log n)$

Binary Recursion: Recurrence

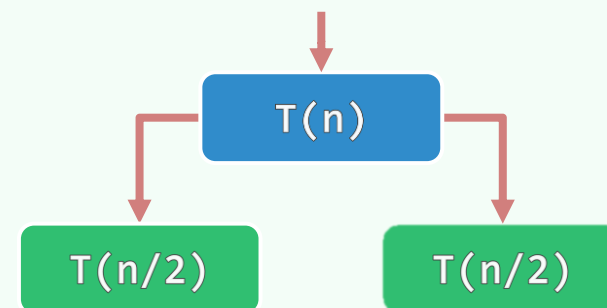
❖ 从递推的角度看，为求解 $\text{sum}(A, \text{lo}, \text{hi})$ ，需要首先

- 递归求解 $\text{sum}(A, \text{lo}, \text{mi})$ 和 $\text{sum}(A, \text{mi}, \text{hi})$ ，进而 $// 2 * T(n/2)$
- 将子问题的解累加 $// O(1)$

❖ 递推方程： $T(n) = 2 \cdot T(n/2) + O(1)$

$$T(1) = O(1) \quad // \text{base: } \text{sum}(A, k, k)$$

❖ 求解：
$$\begin{aligned} T(n) &= 4 \cdot T(n/4) + O(3) = 8 \cdot T(n/8) + O(7) = 16 \cdot T(n/16) + O(15) = \dots \\ &= n \cdot T(1) + O(n-1) = O(2n-1) = O(n) \end{aligned}$$



Master Theorem (1/2)

分治策略对应的递推式，通常形如：

$$T(n) = a \cdot T(n/b) + \mathcal{O}(g(n))$$

//原问题被分为a个规模均为n/b的子任务

//每个递归实例自身，耗时g(n)

❖ 若 $g(n) = \Omega(n^{\log_b a + \epsilon})$, 则 $T(n) = \Theta(g(n))$

- quickSelect (average case): $T(n) = 1 \cdot T(n/2) + \mathcal{O}(n) = \mathcal{O}(n)$

❖ 若 $g(n) = \mathcal{O}(n^{\log_b a - \epsilon})$, 则 $T(n) = \Theta(n^{\log_b a})$

compare >>

- recursive sum: $T(n) = 2 \cdot T(n/2) + \mathcal{O}(1) = \mathcal{O}(n)$

- kd-search: $T(n) = 2 \cdot T(n/4) + \mathcal{O}(1) = \mathcal{O}(\sqrt{n})$

- large integer multiplication: $T(n) = 3 \cdot T(n/2) + \mathcal{O}(n) = \mathcal{O}(n^{\log_2 3}) \dots$

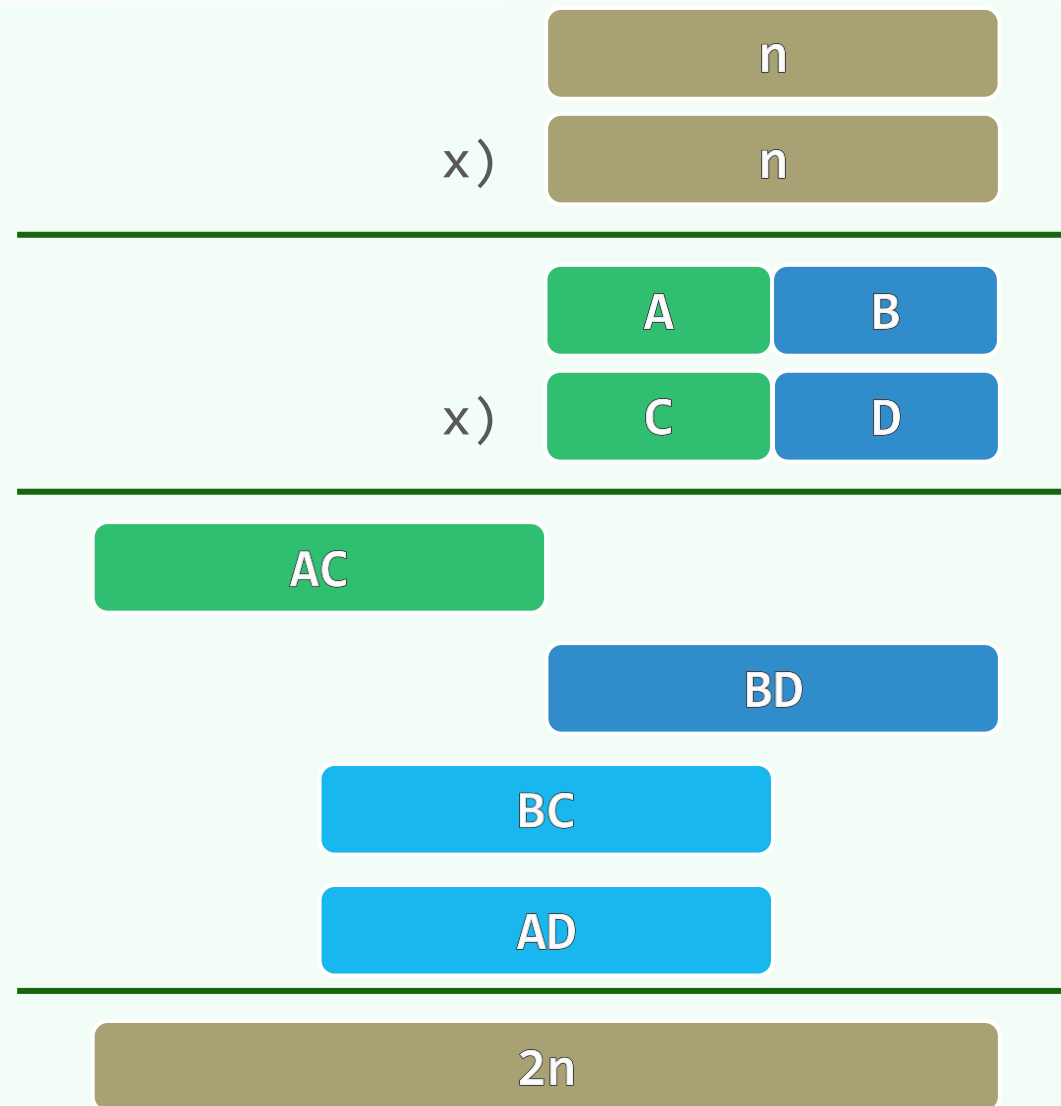
Large Integer Multiplication: Naïve + DAC

$$T(n) = 4 \cdot T(n/2) + \mathcal{O}(n)$$

$$= \mathcal{O}(n^{\log_2 4})$$

$$= \mathcal{O}(n^2)$$

$$B \times C + A \times D = \dots\dots$$



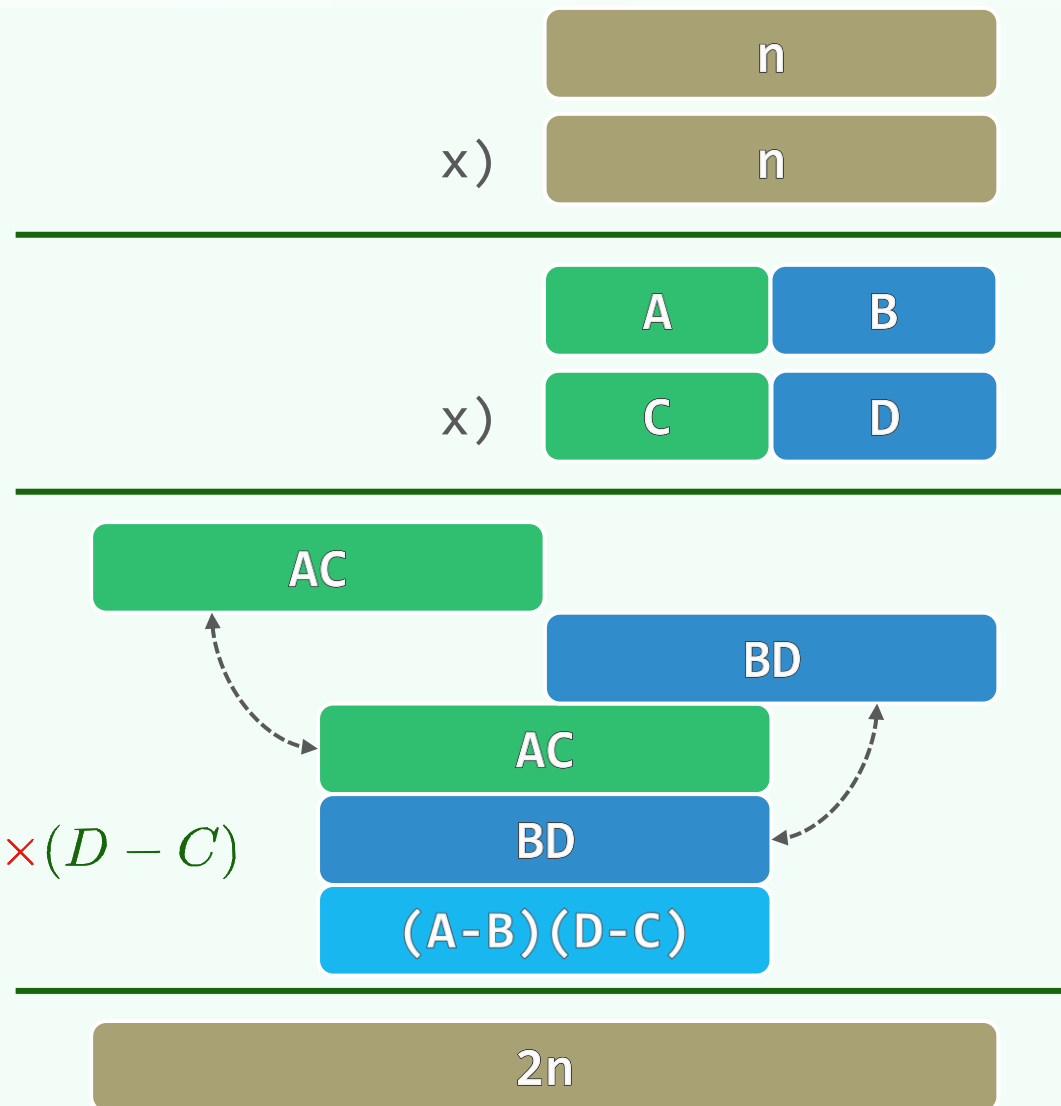
Large Integer Multiplication: Optimal

$$T(n) = 3 \cdot T(n/2) + \mathcal{O}(n)$$

$$= \mathcal{O}(n^{\log_2 3})$$

$$\approx \mathcal{O}(n^{1.585})$$

$$B \times C + A \times D \equiv A \cdot C + B \cdot D + (A - B) \times (D - C)$$



Master Theorem (2/2)

分治策略对应的递推式，通常形如：

$$T(n) = a \cdot T(n/b) + \mathcal{O}(g(n))$$

//原问题被分为a个规模均为n/b的子任务

//任务的划分、解的合并，总共耗时g(n)

❖ 若 $g(n) = \Theta(n^{\log_b a} \cdot \log^k n)$ 且 $0 \leq k$ ，则 $T(n) = \Theta(g(n) \cdot \log n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$

- **binary search:** $T(n) = 1 \cdot T(n/2) + \mathcal{O}(1) = \mathcal{O}(\log n)$

- **mergesort:** $T(n) = 2 \cdot T(n/2) + \mathcal{O}(n) = \mathcal{O}(n \cdot \log n)$

- **STL mergesort:** $T(n) = 2 \cdot T(n/2) + \mathcal{O}(n \cdot \log n) = \mathcal{O}(n \cdot \log^2 n)$

<< compare

❖ 要是...不巧落在...这三种情况之间的**縫隙**呢？

或者...子任务的...规模**不均等**呢？

Akra-Bazzi Theorem

分治策略对应的递推式，更一般地形如：

//k组：各含 a_i 个规模为 $b_i n + h_i(n)$ 的子任务

$$T(n) = \sum_{i=1}^k a_i \cdot T(b_i \cdot n + h_i(n)) + \mathcal{O}(g(n)) \quad // \text{任务的划分、解的合并，总共耗时 } g(n)$$

- $a_i > 0$, $0 < b_i < 1$, $|h_i(n)| = \mathcal{O}(n / \log^2 n)$

- $g(n) \geq 0$, 且有常数 $d > 0$ 使得 $|g'(n)| = \mathcal{O}(n^d)$ //多项式增长

❖ 只要取 p 使得 $\sum_{i=1}^k a_i \cdot b_i^p = 1$, 则 $T(n) = \Theta(n^p \cdot (1 + \int_1^n g(u)/u^{p+1} du))$ //与 $h_i(n)$ 无关

✖ **坏选择**： $T(n) = 1 \cdot T(\underline{3/4} \cdot n) + 1 \cdot T(\underline{1/4} \cdot n) + \mathcal{O}(n) = \mathcal{O}(n \log n)$ //p=1 |linearSelect

✓ **好选择**： $T(n) = 1 \cdot T(\underline{3/4} \cdot n) + 1 \cdot T(\underline{1/5} \cdot n) + \mathcal{O}(n) = \mathcal{O}(n)$ //p<1 |算法 (第14章)

❖ $T(n) = \sum_{k=1}^9 T(k \cdot n)/2025 + \sqrt[2025]{\prod_{k=3}^{12} n^{k^3}} = \mathcal{O}(n^3 \cdot \log n)$

绪论

迭代与递归：总和最大区间

01-ES

这时，公共智慧的结果便产生理智与意志在社会体中的结合，
也才有了各个部分的密切配合，以及最后全体的最大力量

邓俊辉

deng@tsinghua.edu.cn

问题 + 蛮力算法

❖ 从整数序列中，找出总和最大的区间（有多个时，**短者、靠后者**优先）

$\mathcal{A}[0, 19) = \{ 1, -2, \underline{7, 2, 6, -9, 5, 6}, -12, -8, \underline{13, 0, -3, 1, -2, 8, 0}, -5, 3 \}$

```
int gs_BF( int A[], int n ) //蛮力策略:  $\mathcal{O}(n^3)$ 
```

```
    int gs = 0; //目前已知的最大和: 零宽度区间, 总和为零
```

```
    for ( int lo = 0; lo < n; lo++ ) //枚举所有的
```

```
        for ( int hi = lo+1; hi <= n; hi++ ) // $\mathcal{O}(n^2)$ 个区间!
```

```
            int s = sum( A, lo, hi ); //用 $\mathcal{O}(n)$ 时间求和
```

```
            gs = max( gs, s ); //择优更新
```

```
    return gs;
```



递增策略

❖ 考查所有起点**相同**的区间...

它们的总和之间具有**相关性**...

```
int gs_IC( int A[], int n ) //递增策略:  $O(n^2)$ 

    int gs = 0; //目前已知的最大和: 零宽度区间, 总和为零

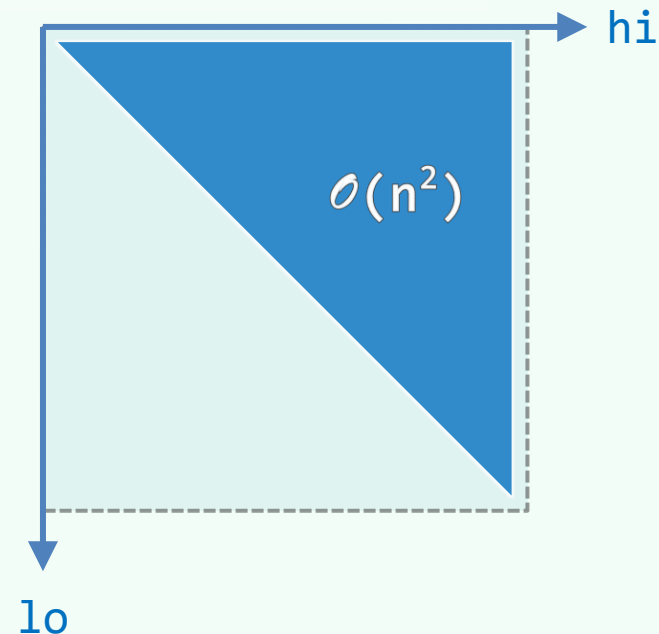
    for ( int lo = 0; lo < n; lo++ ) //枚举所有起始于lo

        for ( int s = 0, hi = lo; hi < n; ) //终止于hi的区间

            s += A[ hi++ ]; //递增地得到其总和:  $O(1)$ 

            gs = max( gs, s ); //择优更新

    return gs;
```



分而治之：前缀 + 后缀： $\mathcal{A}[lo, hi) = \mathcal{A}[lo, mi) \cup \mathcal{A}[mi, hi) = \mathcal{P} \cup \mathcal{S}$

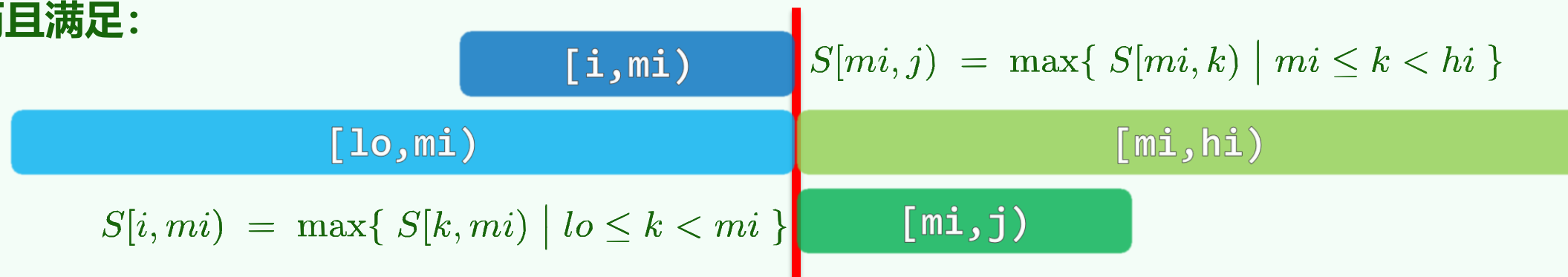
❖ 借助递归，便可求得 $\mathcal{P}$ 、 $\mathcal{S}$ 内部的GS；而剩余的**实质**任务无非是...

考察那些**跨越**切分线的区间...

❖ 沿着切分线，这类区间必被分割为**非空**的前缀、后缀：

$$\mathcal{A}[i, j) = \mathcal{A}[i, mi) + \mathcal{A}[mi, j), \quad i < mi < j$$

而且满足：



❖ 可见，二者均可独立计算，且累计耗时不过 $\mathcal{O}(n)$ ；于是总体复杂度也优化为 $\mathcal{O}(n \cdot \log n)$

分而治之：实现

```
int gs_DC( int A[], int lo, int hi ) { //Divide-And-Conquer:  $\mathcal{O}(n \cdot \log n)$ 

    if ( hi - lo < 2 ) return A[lo]; //递归基

    int mi = (lo + hi) / 2; int gsL = 0, gsR = 0; //前缀 | 后缀中的最大后缀 | 前缀

    for ( int s = 0, i = mi-1; lo <= i; i-- ) //枚举
        gsL = max( gsL, s += A[i] ); //所有[i,mi)

    for ( int s = 0, j = mi; j < hi; j++ ) //枚举
        gsR = max( gsR, s += A[j] ); //所有[mi,j)

    return max( gsL + gsR, max( gs_DC(A, lo, mi), gs_DC(A, mi, hi) ) ); //递归

} //gs_DC
```

减而治之：原理

❖ 关注GS的**终点**，考查每一个可能的 $n \dots$

❖ 由短渐长，考查所有的后缀 $[k, n)$

视 $S[k, n)$ 的正负，无非两种情况

❖ 若 $S[k, n) > 0$

- 则 k 不可能是GS的**终点**

故可继续考查 $[k-1, n)$

- 当然， $[k, n)$ 可能就是GS：

递增地求和，并择优、更新即可

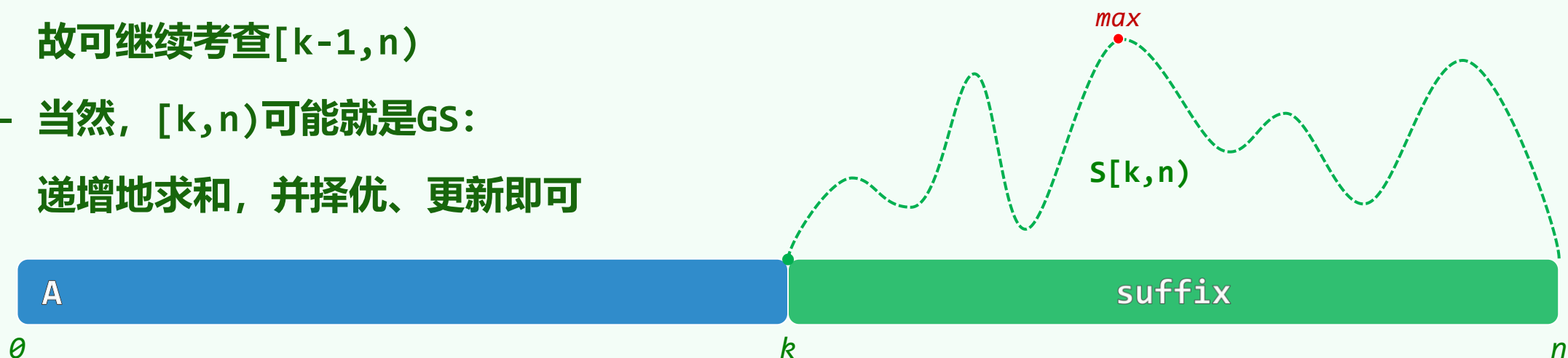
❖ 反之，若 $S[k, n) \leq 0$

- 则更长的后缀不可能是GS

故可剪除 $[k, n)$ —— 令 $n = k$

- 当然， k 仍可能是GS的**终点**：

重复以上，继续考查后缀



减而治之：实现

```
int gs_LS( int A[], int n ) { //减治策略:  $O(n)$ 

    int gs = 0; //目前已知的最大和 =  $S[n,n) = 0$ 

    for ( int sum = 0, k = n; 0 < k-- ) { //当前区间[k,n)

        sum += A[k]; //递增地更新后缀和

        gs = max( gs, sum ); //择优更新

        sum = max( sum, 0 ); //剪除总和为负的后缀

    }

    return gs;

}
```

to scan

$[0,n)$

绪论

动态规划: Fibonacci数

聖人不記事，所以常記得；今人忘事，以其記事

有人建议不妨置备一本签名簿，供来访者留下自己的名字，就像怀特山那样；可是，天哪！我的记性非常好，用不着那个玩意儿

Repetitive diligence is just another form of laziness.

邓俊辉

deng@tsinghua.edu.cn

递归版fib()

$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$$

0 1 1 2 3 5 8 13 21 34 55 89 ...

```
uint64_t fib(n) { return (2 > n) ? n : fib(n-1) + fib(n-2); } //为何这么慢?
```

❖ 复杂度: $T(n) = T(n-1) + T(n-2) + 1$

- 若令 $S(n) = [T(n) + 1]/2$, 则 $S(0) = 1 = \text{fib}(1)$, $S(1) = 1 = \text{fib}(2)$

- 故有 $S(n) = S(n-1) + S(n-2) = \text{fib}(n+1)$

$$T(n) = 2 \cdot S(n) - 1 = 2 \cdot \text{fib}(n+1) - 1 = \mathcal{O}(\phi^n)$$

- 其中 $\phi = (1 + \sqrt{5})/2 \approx 1.618$

❖ 也可以...猜... $T(n) = \mathcal{O}(\alpha^n - 1)$

于是 $\mathcal{O}(\alpha^n) = \mathcal{O}(\alpha^{n-1}) + \mathcal{O}(\alpha^{n-2})$, $\alpha^2 = \alpha^1 + \alpha^0$, 解得 $\alpha = (1 + \sqrt{5})/2 = \phi$

封底估算

$$\phi^{36} \approx 2^{25}$$

$$\phi^{43} = 2^{43 \cdot 25 / 36} = 2^{1075 / 36} \approx 2^{1080 / 36} = 2^{30} \approx 10^9 \text{ flo} \sim 1 \text{ sec}$$

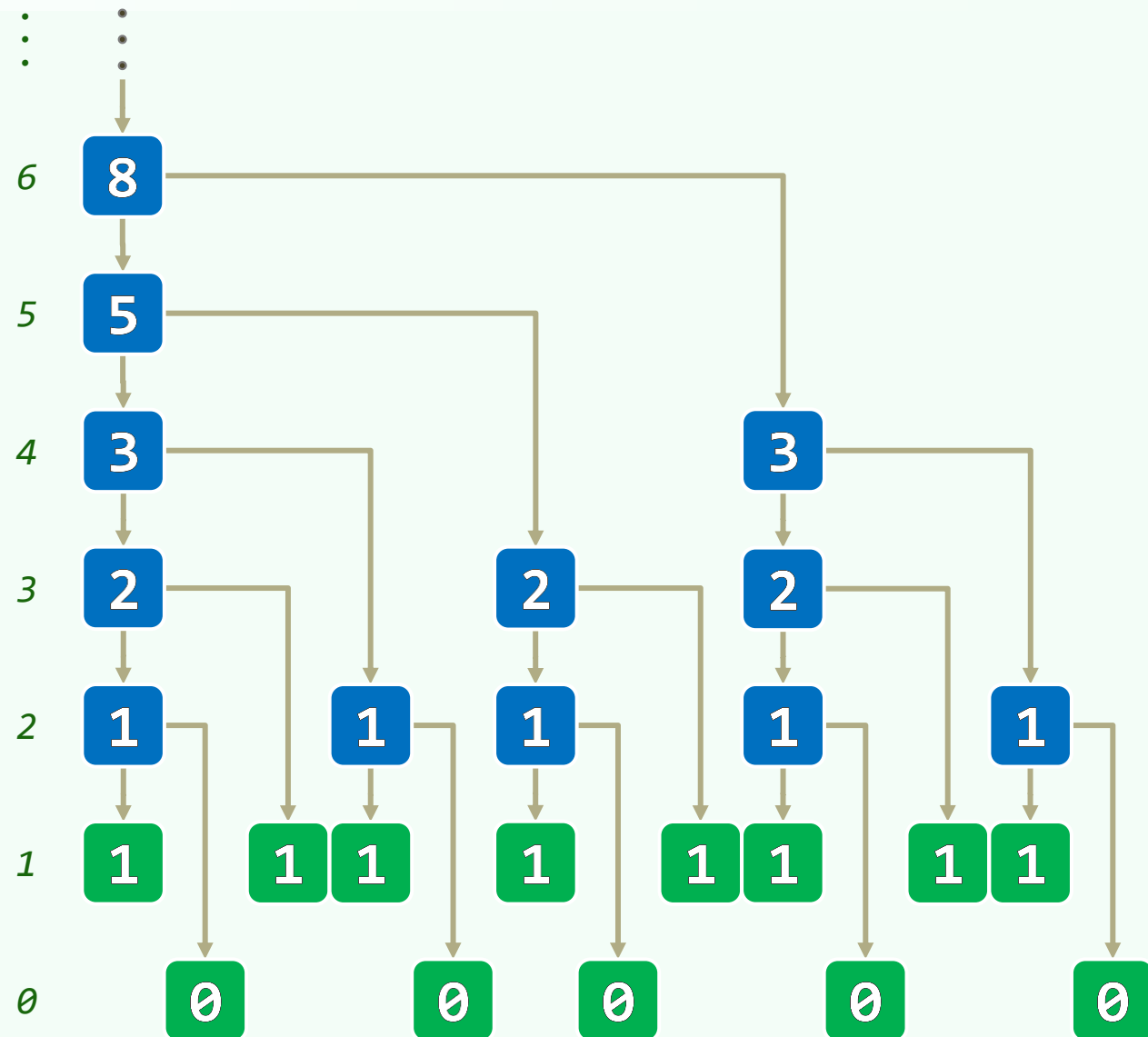
$$\phi^5 \approx 10^1$$

$$\phi^{67} \approx 10^{14} \text{ flo} \sim 10^5 \text{ sec} \approx 1 \text{ day}$$

$$\phi^{92} \approx 10^{19} \text{ flo} \sim 10^{10} \text{ sec} \approx 10^5 \text{ day} \approx 3 \text{ century}$$

Memoization: 原理

- ❖ 低效的根源在于
有大量重复的递归实例
- ❖ 先后出现的实例，共计 $\mathcal{O}(\phi^n)$ 个
而去重之后，总共不过 $\mathcal{O}(n)$ 种
- ❖ 再三出现、结果不变的计算，何必重复？
记住答案，直接CX就好！
- ❖ 好记性不如烂笔头
准备一张速查表...



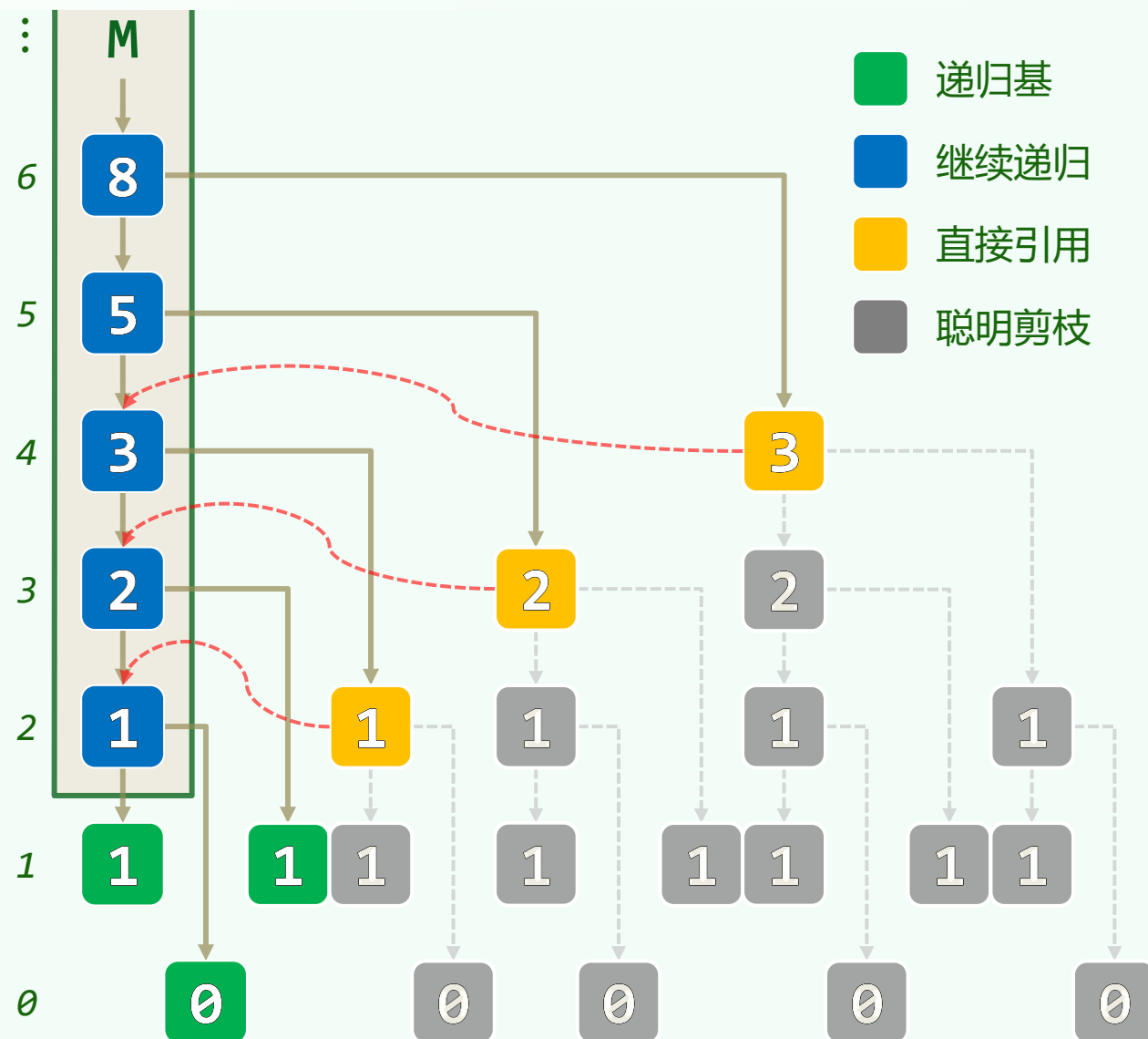
Memoization: 实现

❖ 动手计算此前先查表，如已算过，便**直接引用**

```
extern uint64_t M[]; //初始化为全0
```

❖ 每个子问题**仅算一次**，算罢**随即记入速查表**

```
uint64_t fibM( int n ) { //O(n)
    if ( n < 2 ) return n; //递归基
    if ( !M[n] ) //尚未算出，照常递归
        M[n] = fibM(n-1) + fibM(n-2);
    return M[n]; //直接引用
} //fibM
```



Memoization: 一般模式

```
def f(n)
    if ( n < 1 ) return trivial( n );

    return f(n-X) + f(n-Y)*f(n-Z);
```

```
T M[ N ]; #init. with UNDEFINED
```

```
def f(n)
    if ( n < 1 ) return trivial( n );

    # recur only when necessary &
    # always write down the result
    if ( M[n] == UNDEFINED )
        M[n] = f(n-X) + f(n-Y)*f(n-Z);

    return M[n];
```

Dynamic Programming: 颠倒计算方向

❖ 由自顶而下递归, 改为自底而上迭代

❖ `f = 1; g = 0; //fib(-1), fib(0)`

```
while ( 0 < n-- ) {
```

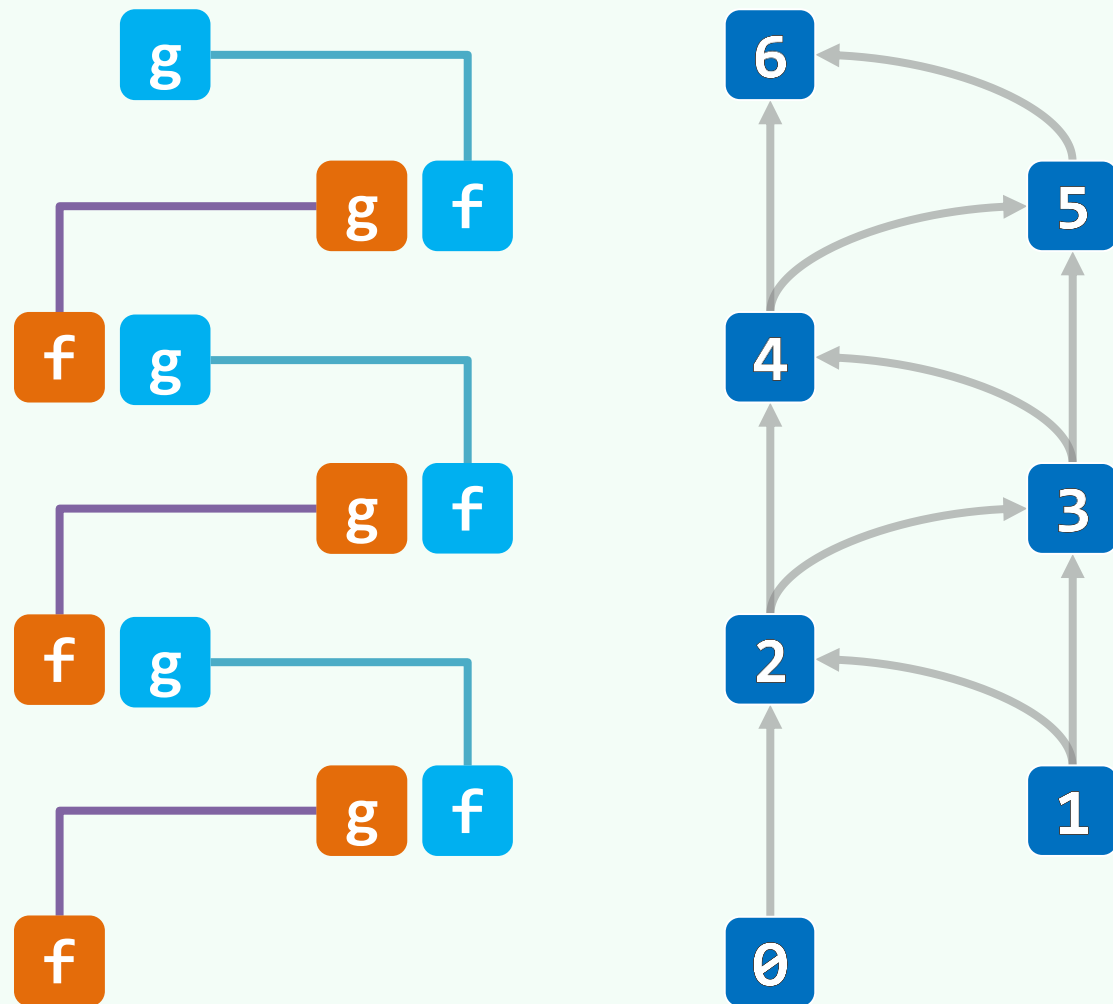
```
    g = g + f;
```

```
    f = g - f;
```

```
}
```

```
return g;
```

❖ $T(n) = O(n)$, 而且仅需 $O(1)$ 空间!



绪论

动态规划：最长公共子序列

01-F2

Make it work, make it right, make it fast.

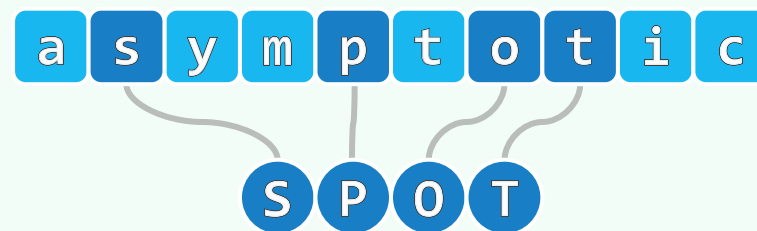
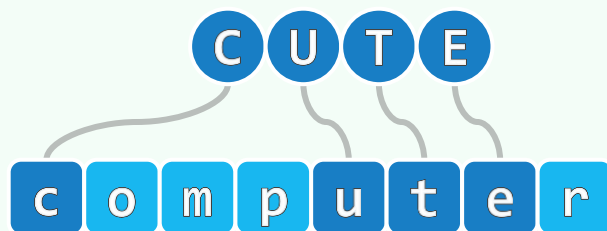
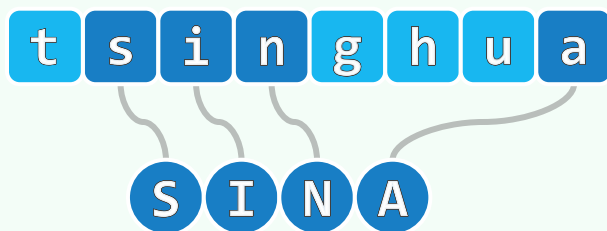
世上一切都无独有偶，为什么你与我却否？

邓俊辉

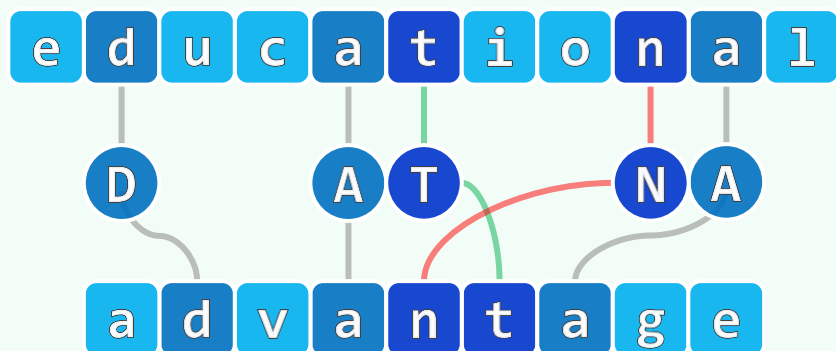
deng@tsinghua.edu.cn

Longest Common Subsequence

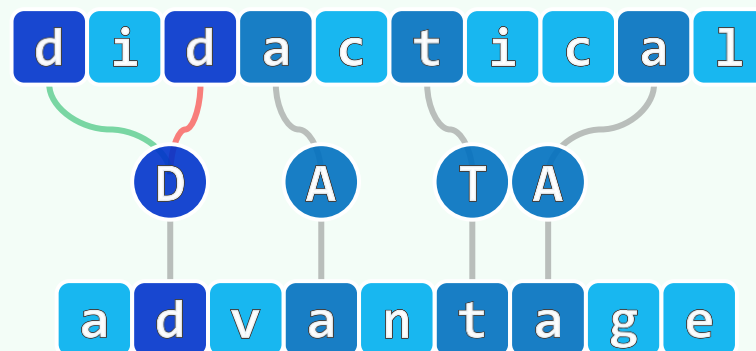
❖ 子序列 (Subsequence) : 由序列中若干字符, 按原相对次序构成



❖ 最长公共子序列: 两个序列之间公共子序列中的最长者

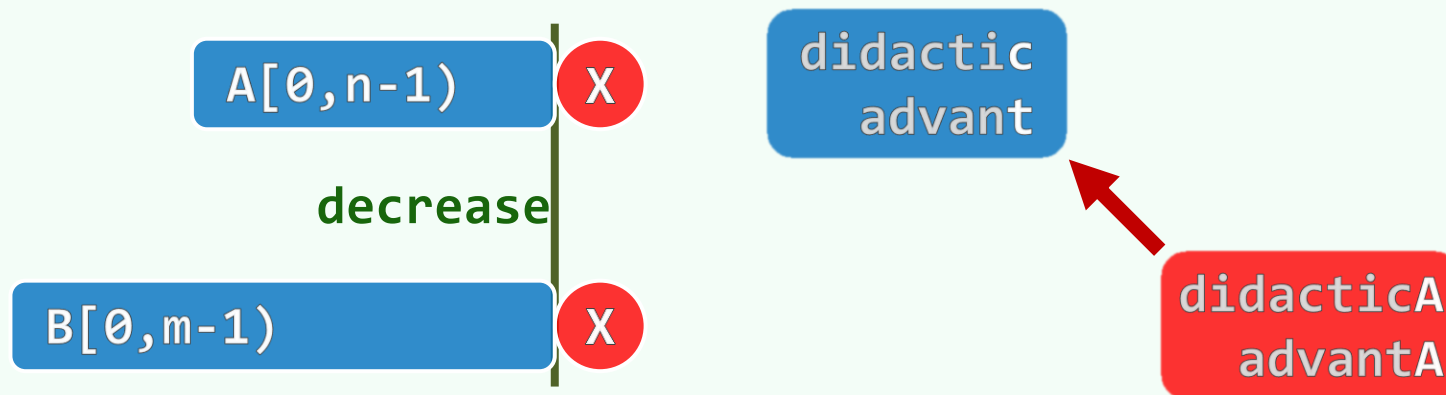


多解



歧义

减治递推

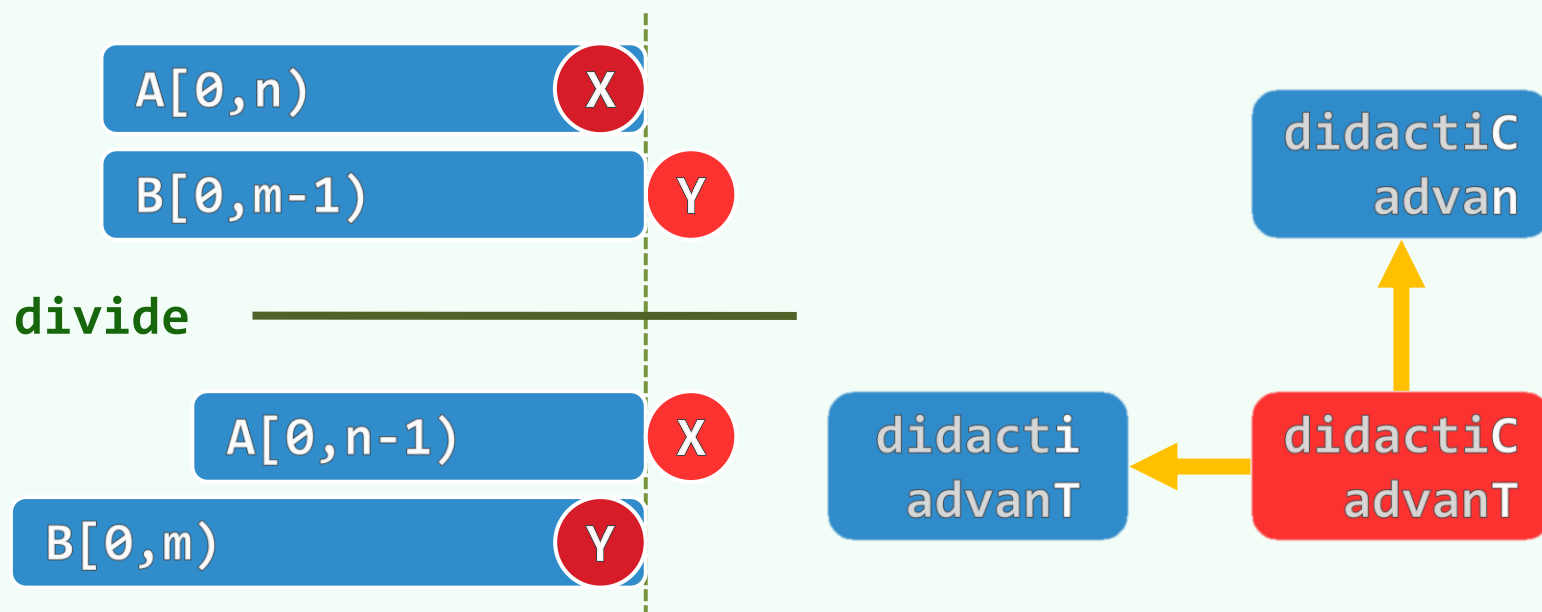


对于序列 $A[0, n)$ 和 $B[0, m)$, $LCS(n, m)$ 无非三种情况

0) 若 $n = 0$ 或 $m = 0$, 则取作空序列 (长度为零) //递归基: 必然总能抵达

1) 若 $A[n-1] = 'X' = B[m-1]$, 则取作: $LCS(n-1, m-1) + 'X'$

分治递推



2) $A[n-1] \neq B[m-1]$, 则在 $LCS(n, m-1)$ 与 $LCS(n-1, m)$ 中取更长者

描述：伪代码

Input: two strings A and B of length n and m resp.,

Output: (the length of) the longest common subsequence of A and B

lcs(A[], n, B[], m)

Compare the last characters of A and B, i.e., A[n-1] and B[m-1]

If A[n-1] = B[m-1]

 Compute $x = \text{lcs}(A, n-1, B, m-1)$ recursively and return $1 + x$

Else

 Compute $x = \text{lcs}(A, n-1, B, m)$ & $y = \text{lcs}(A, n, B, m-1)$ and return $\max(x, y)$

As the recursion **base**, return 0 when either n or m is 0

实现：递归版

```
uint32_t lcs( char const * A, int n, char const * B, int m ) {
```

```
    if ( n < 1 || m < 1 ) //trivial cases
```

```
        return 0;
```

```
    if ( A[n-1] == B[m-1] ) //decrease & conquer
```

```
        return 1 + lcs( A, n-1, B, m-1 );
```

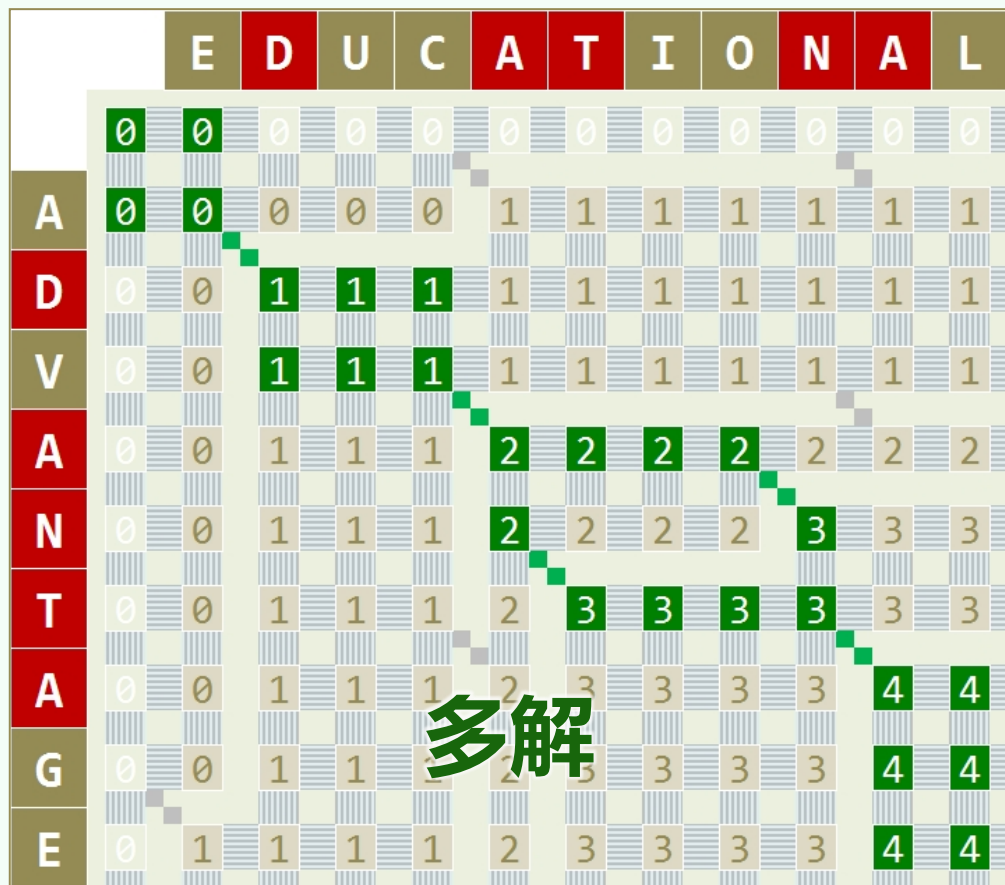
```
    else //divide & conquer
```

```
        return max( lcs( A, n-1, B, m ), lcs( A, n, B, m-1 ) );
```

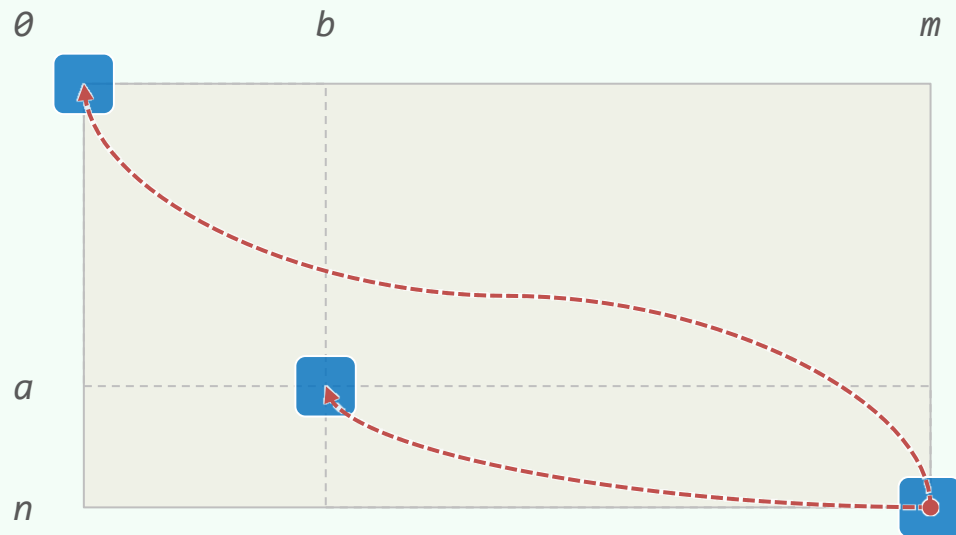
```
}
```

子问题空间

❖ 子问题按递推关系串接的通路，便对应于LCS的解；沿途减治递推的次数，即是LCS长度



指数复杂度



❖ 总共无非 $n \times m$ 种子问题

可见有大量重复的递归

❖ 任何子问题 $\langle a, b \rangle$ ，都可能

重复计算 $\binom{n+m-a-b}{n-a} = \binom{n+m-a-b}{m-b}$ 次

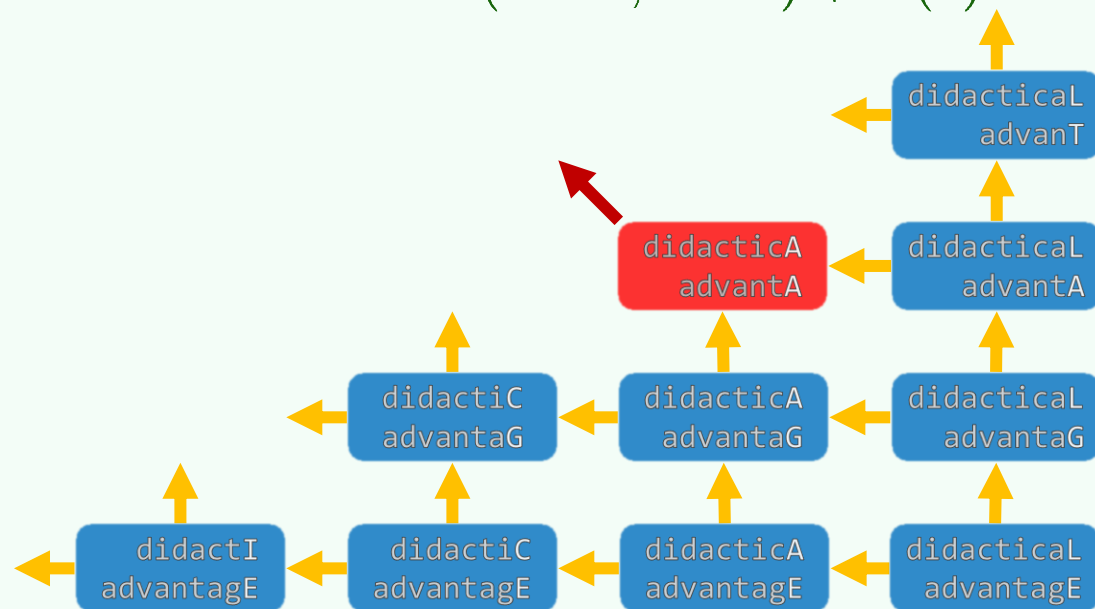
$$\diamond T(n, m) = T(n-1, m-1) + \mathcal{O}(1)$$

$$= \mathcal{O}(\min(n, m))$$

//最好情况

$$\diamond T(n, n) = 2 \cdot T(n, n-1) + \mathcal{O}(1) \quad //最坏情况$$

$$> 4 \cdot T(n-1, n-1) + \mathcal{O}(1) = \Omega(4^n)$$



实现：记忆化版

```
extern uint32_t** M; //lookup table initialized with UINT_MAX

uint32_t lcsM( char const * A, int n, char const * B, int m ) {
    if ( n < 1 || m < 1 ) return 0;
    if ( UINT_MAX == M[n-1][m-1] ) //recurse once for each sub-problem
        M[n-1][m-1] = ( A[n-1] == B[m-1] ) ?
            1 + lcsM( A, n-1, B, m-1 ) :
            max( lcsM( A, n-1, B, m ), lcsM( A, n, B, m-1 ) );
    return M[n-1][m-1];
}
```

❖ 每个子问题至多递归一次，时间复杂度降至 $\mathcal{O}(n \cdot m)$

用到二维的快查表，空间复杂度也悄然增至 $\mathcal{O}(n \cdot m)$ ——如何降低？

动态规划

❖ 翻转思路：从递归基开始，逐步递推

- 以第0行为基础： $LCS(a, 0) = 0$
依次解出第1行的各个子问题： $LCS(a, 1)$
- 以第1行为基础
依次解出第2行的各个子问题： $LCS(a, 2)$
- ...
- 直到 $LCS(n-1, m-1)$

❖ 子问题各计算一次，总共不过 $\mathcal{O}(n \cdot m)$ 时间

❖ 更重要的是，始终只需滚动地记忆两行

于是，空间复杂度随即降至 $\mathcal{O}(\max(n, m))$

❖ 稍作优化，还可进一步降至 $\mathcal{O}(\min(n, m))$

		d	i	d	a	c	t	i	c	a	l
a	0	0	0	0	0	0	0	0	0	0	0
d	0	0	0	0	1	1	1	1	1	1	1
v	0	1	1	1	1	1	1	1	1	1	1
a	0	1	1	1	2	2	2	2	2	2	2
n	0	1	1	1	2	2	2	2	2	2	2
t	0	1	1	1	2	2	3	3	3	3	3
a	0	1	1	1	2	2	3	3	3	4	4
g	0	1	1	1	2	2	3	3	3	4	4
e	0	1	1	1	2	2	3	3	3	4	4

实现：迭代（动态规划）版

```
uint32_t lcs(char const * A, int n, char const * B, int m) {  
  
    if (n < m) { swap(A, B); swap(n, m); } //make sure m <= n  
  
    uint32_t* lcs1 = new uint32_t[m+1]; //the current two rows are  
    uint32_t* lcs2 = new uint32_t[m+1]; //buffered alternatively  
    memset( lcs1, 0x00, sizeof(uint32_t) * (m+1) ); lcs2[0] = 0; //sentinels  
  
    for ( int i = 0; i < n; swap( lcs1, lcs2 ), i++ )  
        for ( int j = 0; j < m; j++ )  
            lcs2[j+1] = ( A[i] == B[j] ) ? 1 + lcs1[j] : max( lcs2[j], lcs1[j+1] );  
  
    uint32_t solu = lcs1[m]; delete[] lcs1; delete[] lcs2; return solu;  
  
}
```